RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 Fundamentals of Data Science
Laboratory Record Note Book

Name : ……

Year / Branch / Section : ……

University Register No. : ……

College Roll No. : ……

Semester : ……

Academic Year : ……

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals of Data Science

NAME	
ROLL NO	
YEAR	
SEC	

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……

Academic Year: …… Semester: …… Branch: ……

Register No. | |

Certified that this is the bonafide record of work done by the above student in the…… Laboratory during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

Sl NO	EXPERIMENT	DATE	GITHUB

Sl NO	EXPERIMENT	DATE	GITHUB

Sl NO	EXPERIMENT	DATE	GITHUB

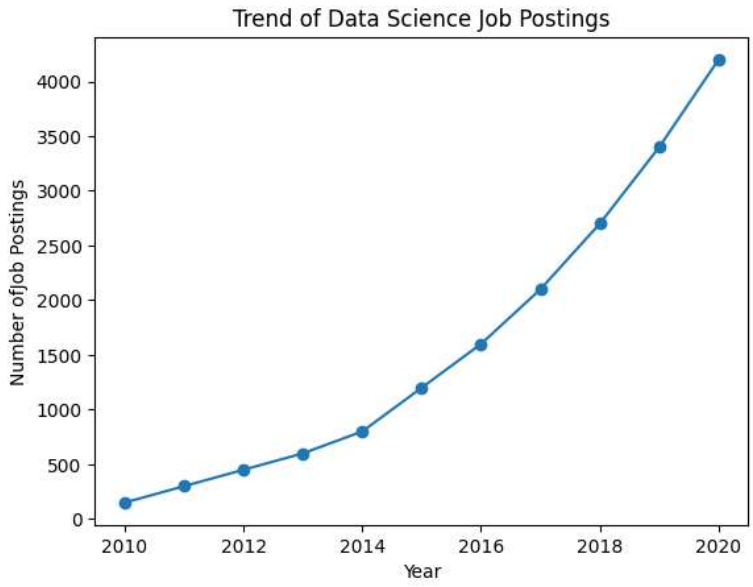
Exp No:1.a Analyze the trend of data science job pstings over the last

Decade

Program

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}
df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```

Output:

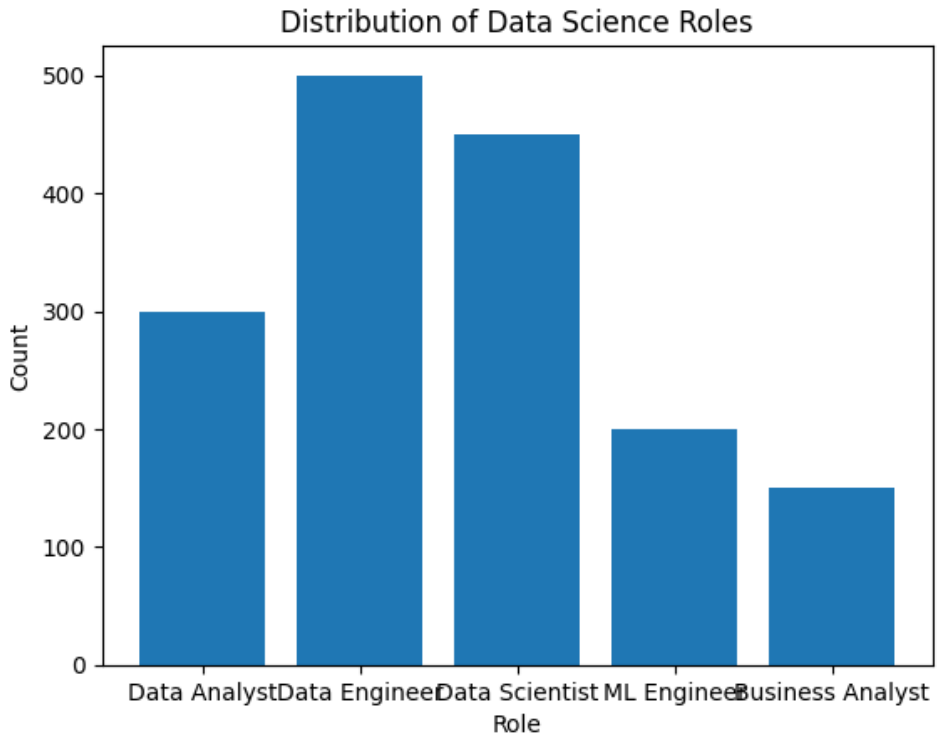


Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Program:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
'Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Output:



Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Program:

```
structured_data = pd.DataFrame({
'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35]})
print("Structured Data:\n", structured_data)
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image, or a video file."
print("\nUnstructured Data:\n", unstructured_data)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

Output:

Structured Data:

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Charlie	35

Unstructured Data:

This is an example of unstructured data. It can be a piece of text, an image, or a video file.

Semi-structured Data:

{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

Program:

```
Import pandas as pd
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

Output:

Original Data: b'Rajalakshmi Engineering College.'

Encrypted Data:
b'gAAAAABqhcTfSFXptI6tdSAIY8CnKnJPSqUD13iwCoizn9igQ__AcAGeU6z7sX6kApZ-oqMc4O6iLerbAG0nItzpfpc8ngB78Ppx__tmdffJXP8XNOb-Ksl9T0VBT0QD8kqw23RX4AP1'

Decrypted Data: b'Rajalakshmi Engineering College.

Exp No:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded=files.upload()
df=pd.read_csv('SALES DATA - SALES DATA.csv')
print(df.head())
print(df.isnull().sum())
df['Sales']=(df['Sales'].fillna(df['Sales'].mean))
df.dropna(subset=['Product','Quantity','Region'],inplace=True)
print(df.describe())
product_summary=df.groupby('Product').agg({
    'Sales':'sum',
    'Quantity':'sum'
}).reset_index()
print(product_summary)
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date']=pd.to datetime(df['Date'],format='%d-%m-%Y')
sales_over_time=df.groupby('Date').agg({'Sales':'sum'}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales over time['Date'],sales over time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()
pivot_table=df.pivot_table(values='Sales',index='Region',columns='Produ
ct',aggfunc='sum',fill value=0)
print(pivot_table)
correlation_matrix=df.select_dtypes(include='number').corr()
print(correlation matrix)
import seaborn as sns
plt.figure(figsize=(10,6))
sns.heatmap(correlation matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

	Date	Product	Sales	Quantity	Region
0	01-01-2023	Product A	200	4	North
1	02-01-2023	Product B	150	3	South
2	03-01-2023	Product A	220	5	North
3	04-01-2023	Product C	300	6	East
4	05-01-2023	Product B	180	4	West

Date 0

Product 0

Sales 0

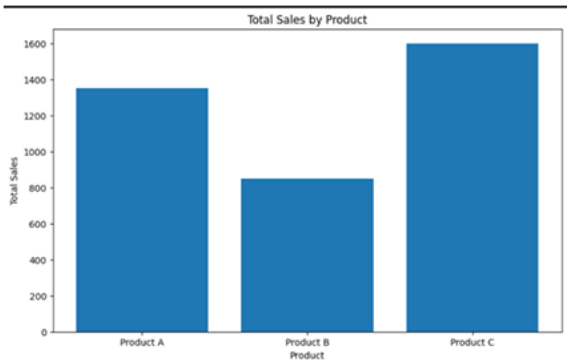
Quantity 0

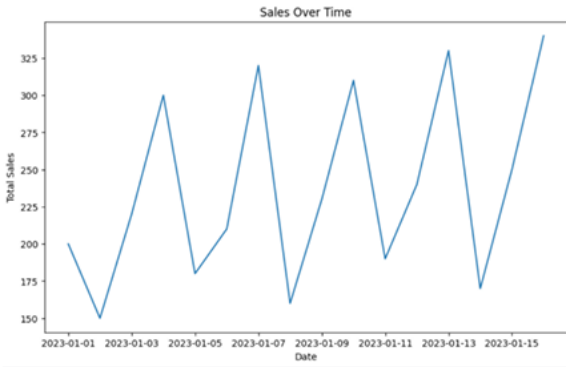
Region 0

dtype: int64

	Sales	Quantity
count	16.000000	16.000000
mean	237.500000	5.375000
std	64.031242	1.746425
min	150.000000	3.000000
25%	187.500000	4.000000
50%	225.000000	5.500000
75%	302.500000	7.000000
max	340.000000	8.00000

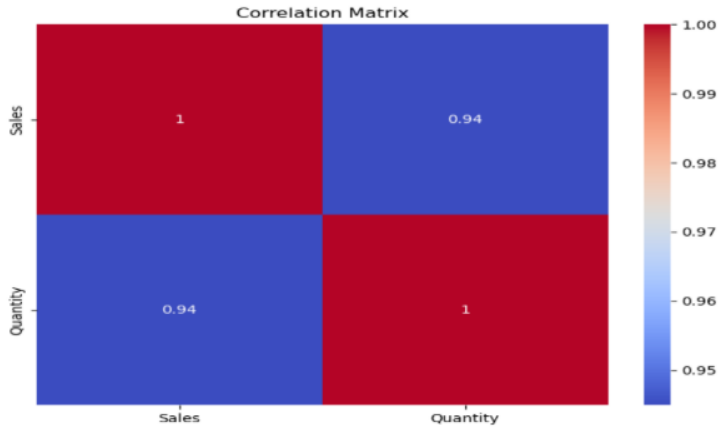
	Product	Sales	Quantity
0	Product A	1350	33
1	Product B	850	17
2	Product C	1600	36





Product	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

	Sales	Quantity
Sales	1.000000	0.944922
Quantity	0.944922	1.000000

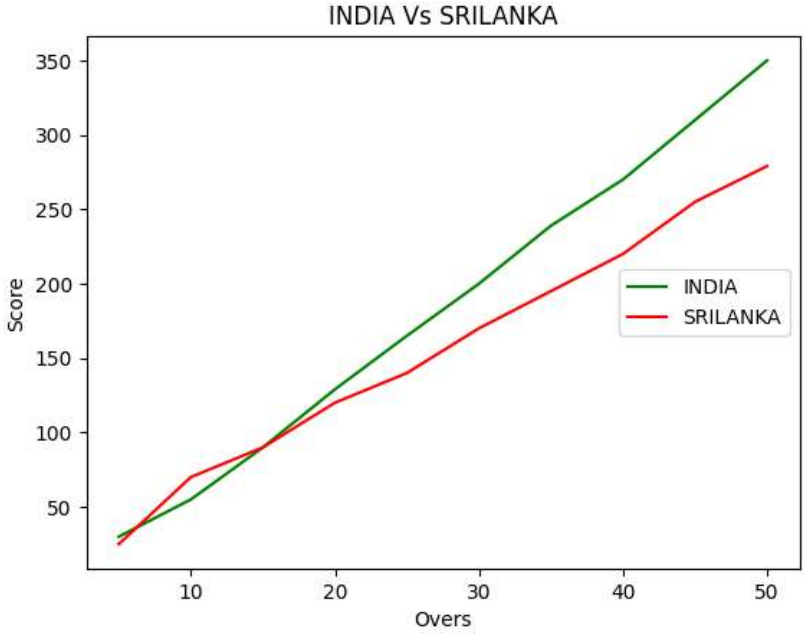


3 a) Conduct an experiment to show data visualization using line plot.

Program:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

Output:



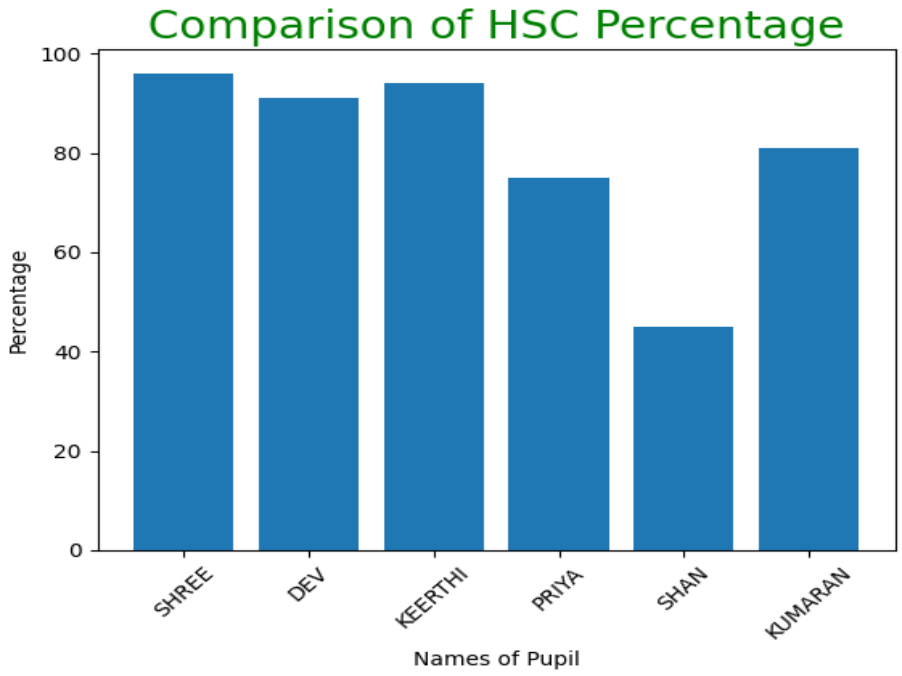
3 b)

Conduct an experiment to show data visualization using bar chart.

Program:-

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
 Percentage_hsc = [96, 91, 94, 75, 45, 81]
 hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
 hscmark.ylabel("Percentage")
 hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output:

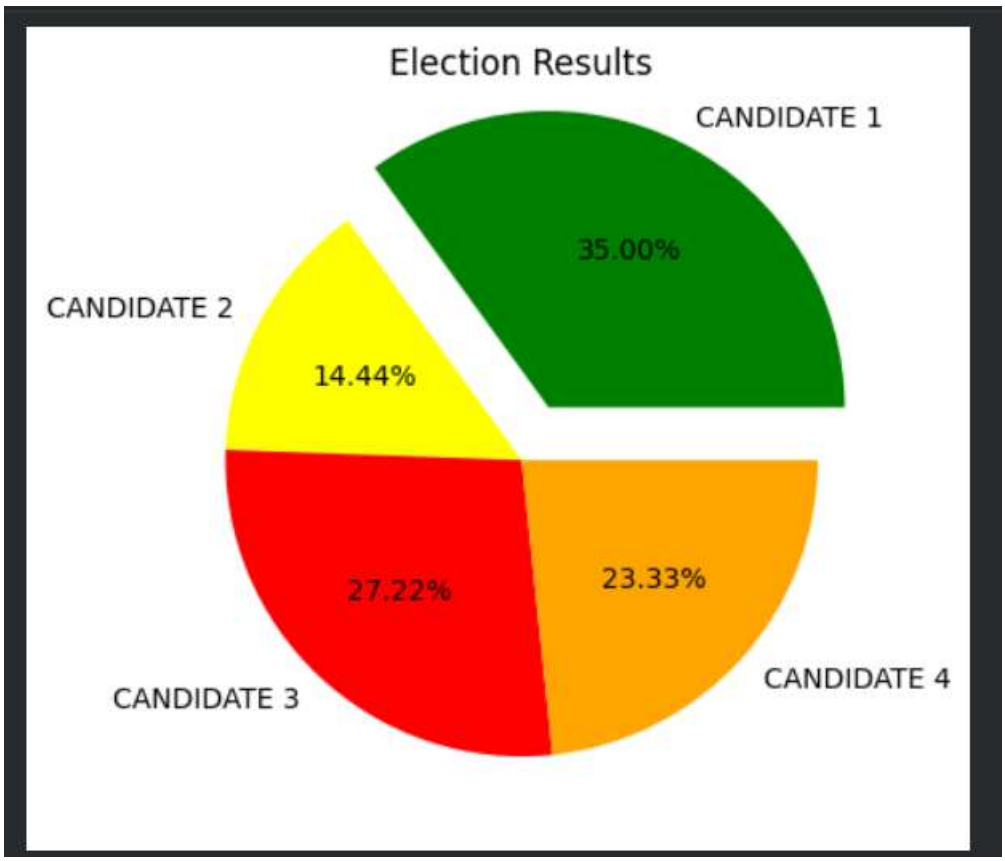


Exp No:3.c Conduct an experiment to show data visualization using pie chart

Program:

```
import matplotlib.pyplot as election
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

Output:



Exp No:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

Exp No:5 Data Collection and Initial Exploration

Program:

1.Data Collection:

```
import pandas as pd
db=pd.read_csv("diabetess.csv")
print(db)
```

Output:

```
     Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]
```

2.Initial Exploration

```
print(db.info())

print(db.describe())

import matplotlib.pyplot as plt

 import seaborn as sns

db.hist(bins=50,figsize=(20,15))

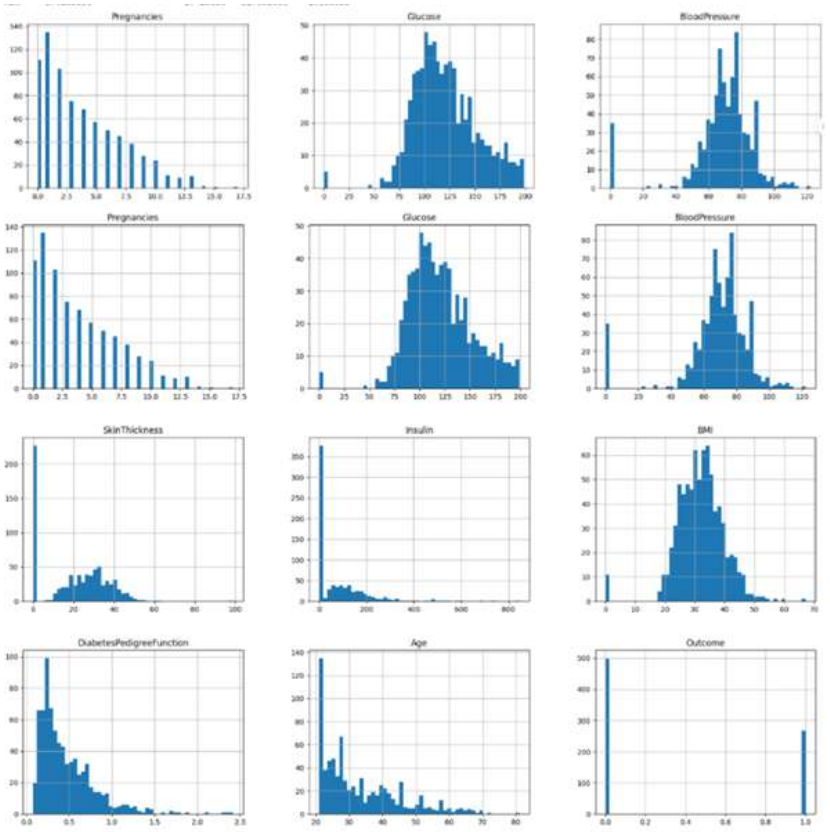
 plt.show()

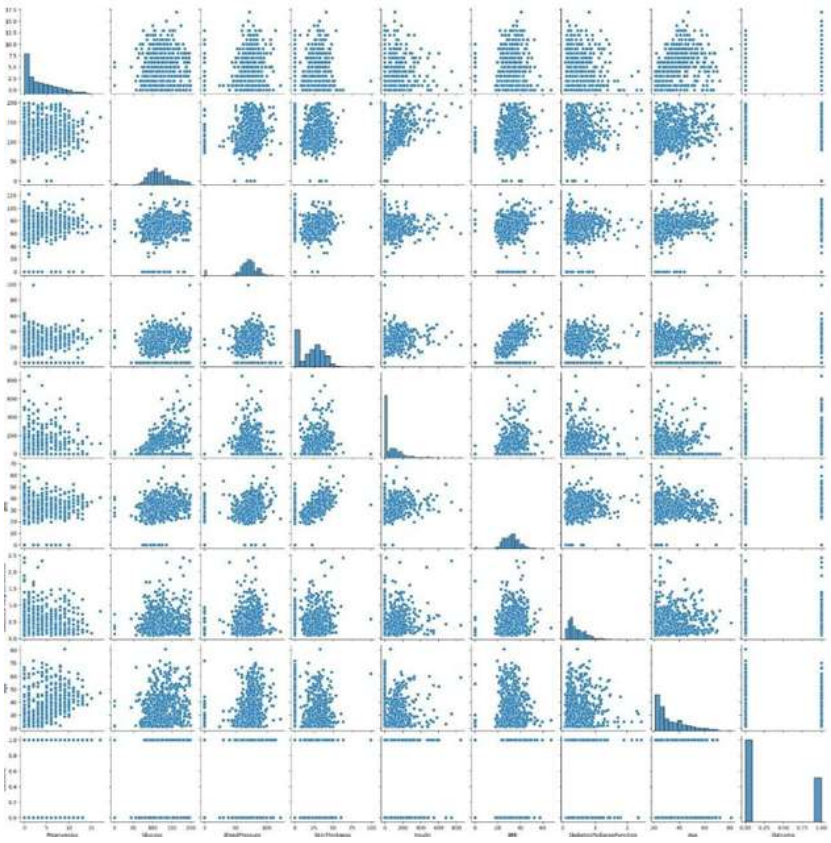
 sns.pairplot(db)

 plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

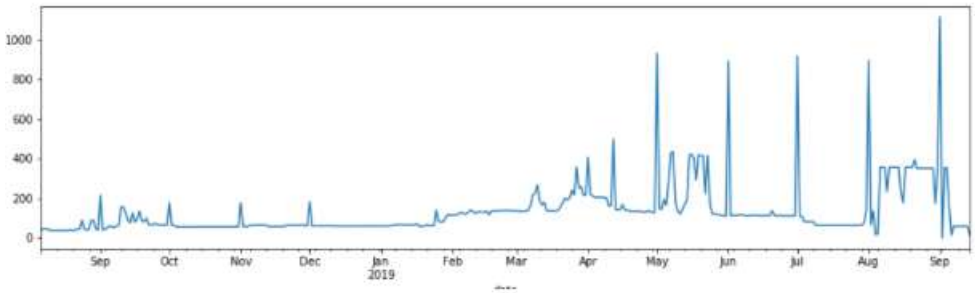


Exp 6: Time Series Analysis of Website Traffic

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
23
plt.show()
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

Output:



Exp 7: Random Sampling and Sampling Distribution

Program:

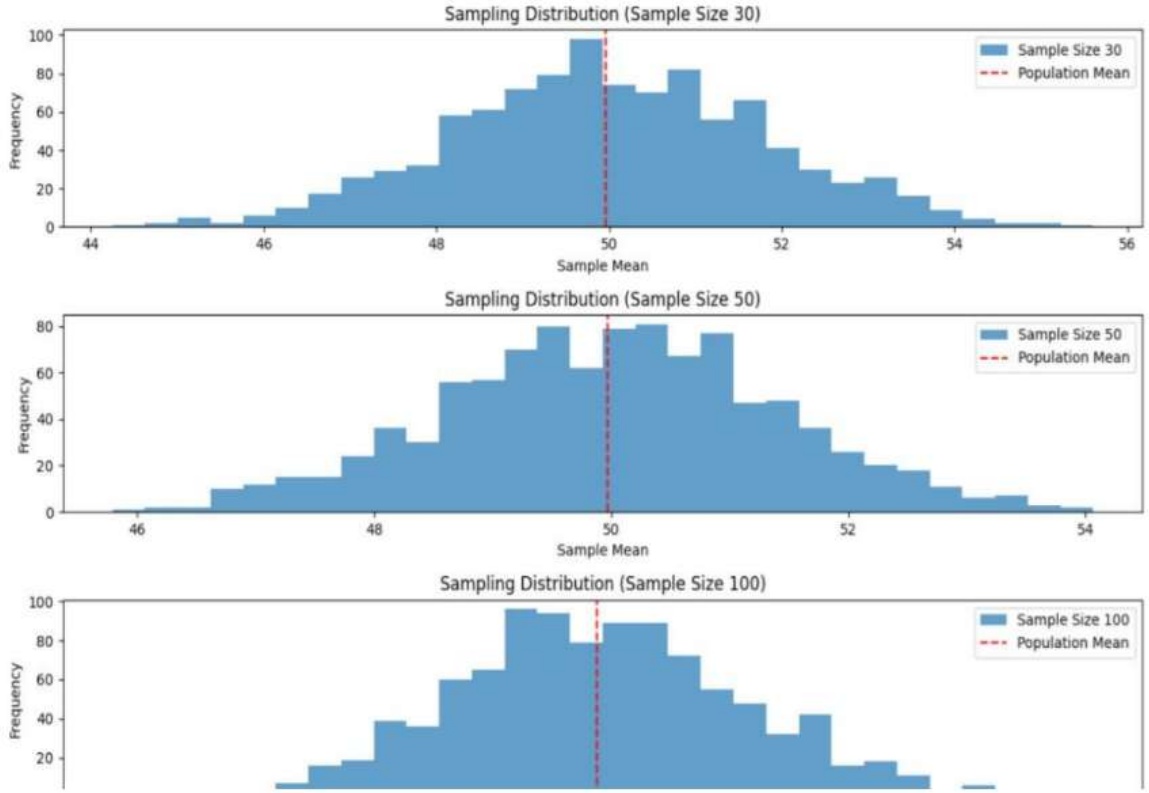
```
import numpy as np
import matplotlib.pyplot as plt
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)
sample_sizes = [30, 50, 100]
num_samples = 1000
26
sample_means = {}

for size in sample_sizes:
  sample_means[size] = []
  for _ in range(num_samples):
    sample = np.random.choice(population, size=size, replace=False)
    sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
  plt.subplot(len(sample_sizes), 1, i+1)
  plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
  plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5, label='PopulationMean')
  plt.title(f'Sampling Distribution (Sample Size {size})')
  plt.xlabel('Sample Mean')
  plt.ylabel('Frequency')
  plt.legend()

plt.tight_layout()
plt.show()
```

Output:


Sampling Distribution (Sample Size 30)
Frequency
Sample Mean
Sample Size 30
Population Mean
Sampling Distribution (Sample Size 50)
Frequency
Sample Mean
Sample Size 50
Population Mean
Sampling Distribution (Sample Size 100)
Frequency
Sample Size 100
Population Mean

Exp 8: Conduct Z test for the Data Given using Python Objective:

To test whether the average weight of a species of birds differs from 150 grams.

Program:

```
import numpy as np
import scipy.stats as stats
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151, 150, 149, 152, 151,
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)

n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
  print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.")
```

Output:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

Exp No:9 Experiment to understand Matplotlib library use cases in Data Science through visualization

Program:

```
Import pandas as pd
Import numpy as np
Import seaborn as sns
Import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv(‘/Iris_Dataset.csv’)
data.info()
data.describe()
data.value_counts(‘Variety)
sns.countplot(x='variety',data=data,)
plt.show()
FinalDataset.head()
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data,)
sns.scatterplot(x='PetalLengthCm',y='PetalWidthCm',hue='variety',data=data,)
sns.pairplot(data,hue='variety',height=3);
plt.show()
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalLengthCm').add_legend();
plt.show();
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalWidthCm').add_legend();
plt.show();
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalLengthCm').add_legend();
plt.show();
```

sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalWidthCm').add_legend();

plt.show();

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv('/Iris_Dataset.csv')
data
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	variety
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
data.describe()
```

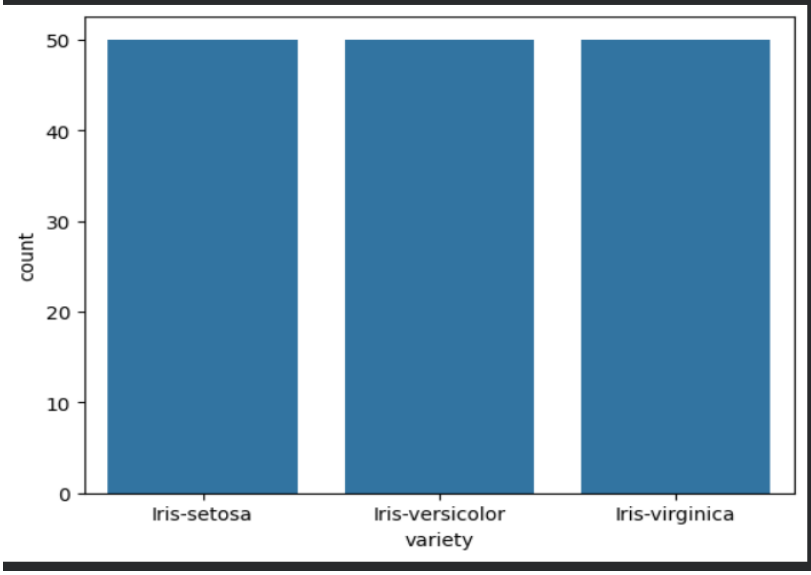
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.057333	3.758000	1.199333
std	43.445368	0.828066	0.435866	1.765298	0.762238
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
data.value_counts('variety')
```

variety	count
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

dtype: int64

```
sns.countplot(x='variety',data=data,)
plt.show()
```

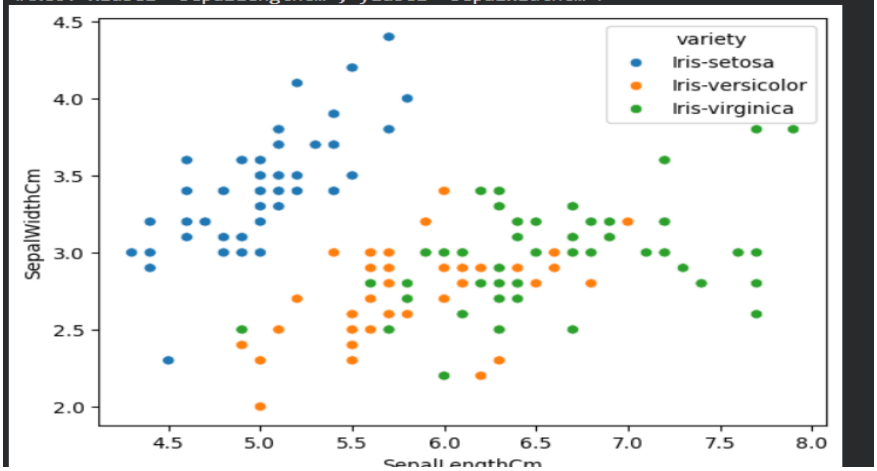


```
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axis=1)
FinalDataset.head()
```

	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm
0	True	False	False	1	5.1	3.5	1.4
1	True	False	False	2	4.9	3.0	1.4
2	True	False	False	3	4.7	3.2	1.3
3	True	False	False	4	4.6	3.1	1.5
4	True	False	False	5	5.0	3.6	1.4

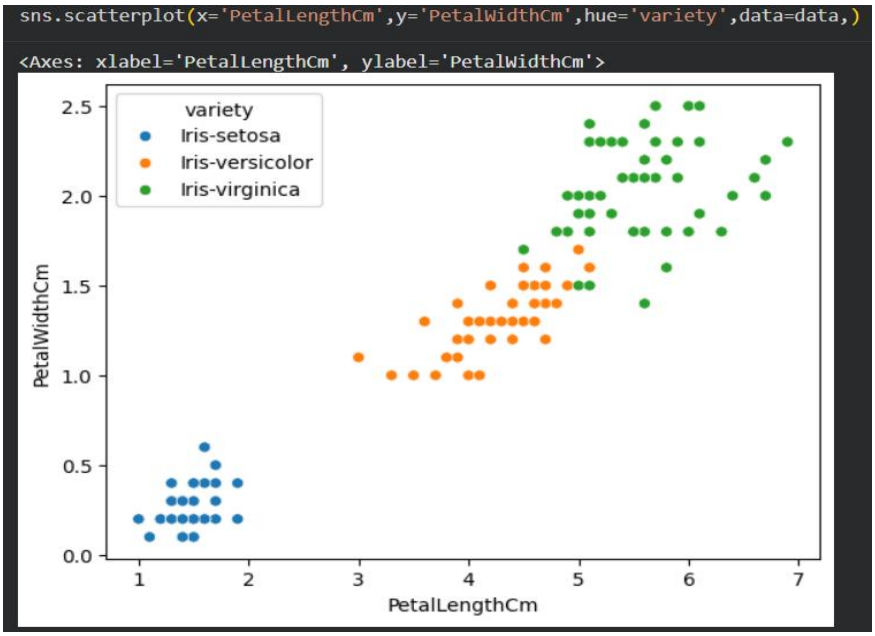
```
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data,)
```

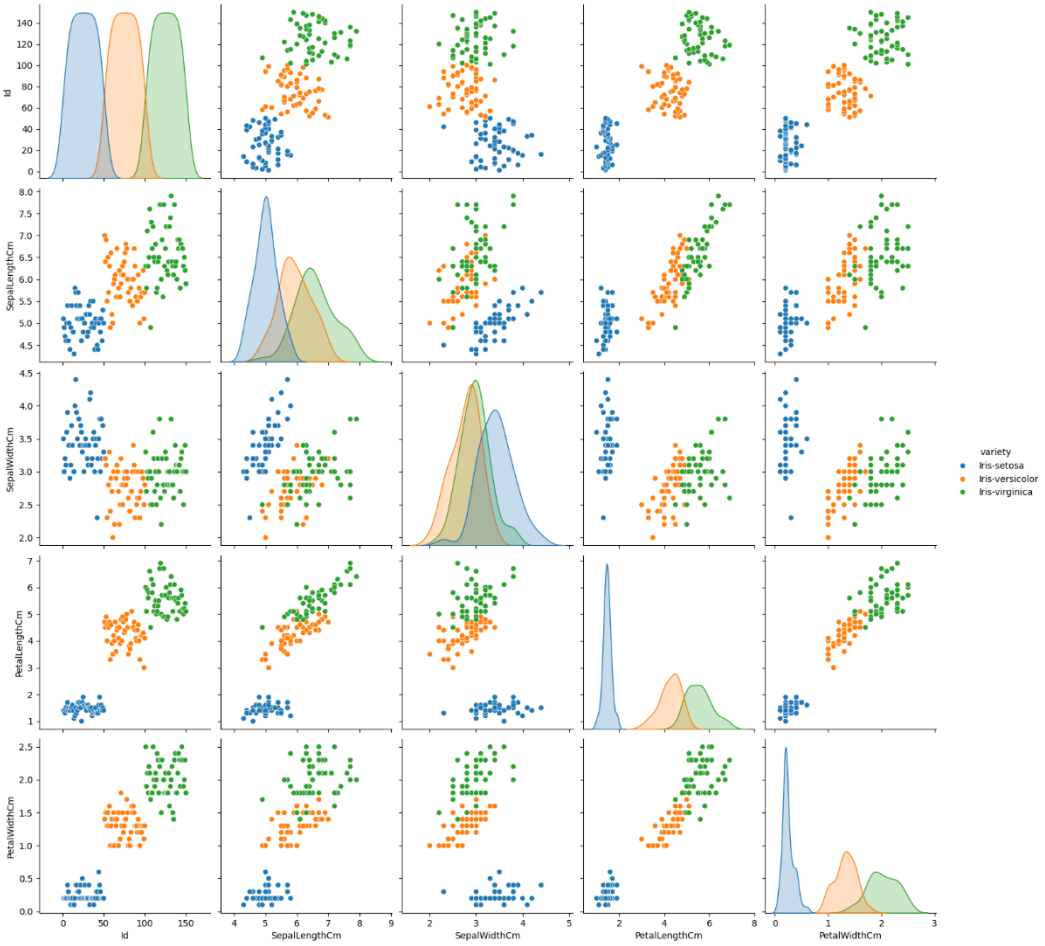
```
<Axes: xlabel='SepalLengthCm', ylabel='SepalWidthCm'>
```



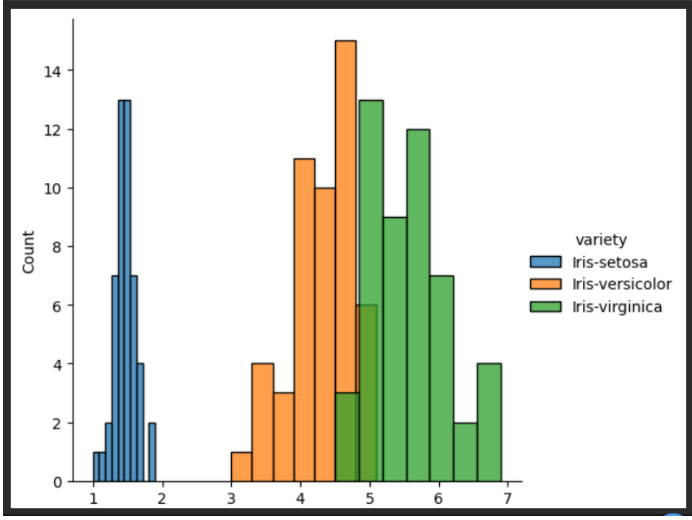

```
sns.scatterplot(x='PetalLengthCm',y='PetalWidthCm',hue='variety',data=data,)
```

```
<Axes: xlabel='PetalLengthCm', ylabel='PetalWidthCm'>
```

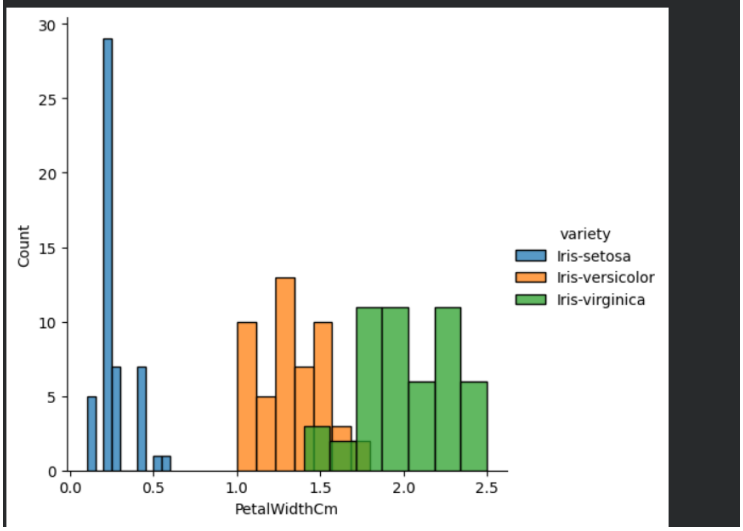




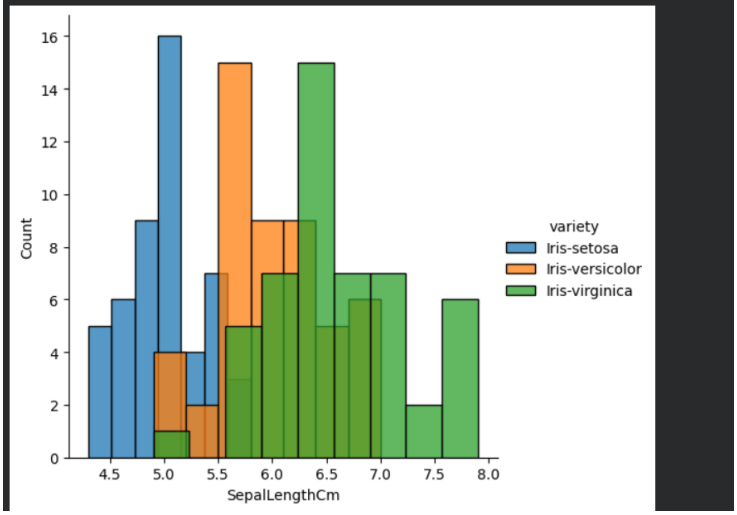

```
plt.show()
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalLengthCm').add_legend();
plt.show();
```



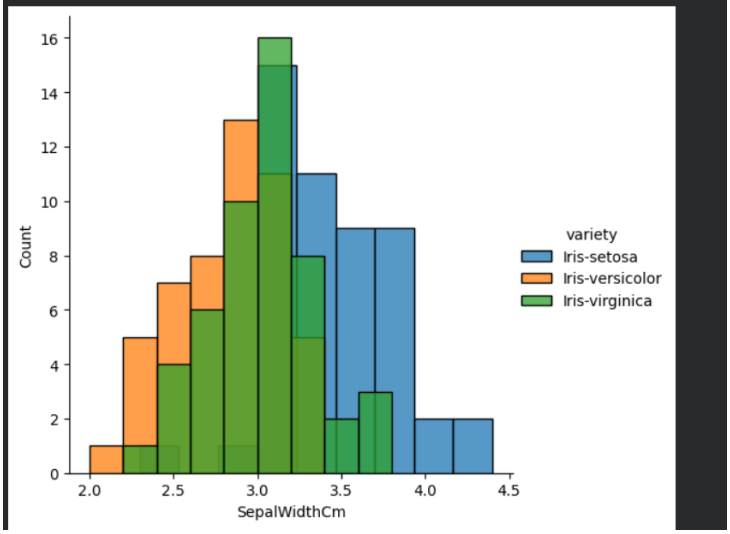
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalWidthCm').add_legend();
plt.show();
```




```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalLengthCm').add_legend();
plt.show();
```




```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalWidthCm').add_legend(
plt.show();
```



Exp No:10

Experiment to understand array function in Data science.

Program:

import numpy as np

array=np.random.randint(1,100,9)

Output:

```
import numpy as np
array=np.random.randint(1,100,9)
array

array([13, 37, 42, 20, 75, 89, 84,  8, 71])
```

np.sqrt(array)

array.ndim

Output:

```
np.sqrt(array)

array([3.60555128, 6.08276253, 6.4807407 , 4.47213595, 8.66025404,
       9.43398113, 9.16515139, 2.82842712, 8.42614977])

array.ndim

1
```

new_array=array.reshape(3,3)

new_array

new_array.ravel()

Output:

```
new_array=array.reshape(3,3)
new_array

array([[13, 37, 42],
       [20, 75, 89],
       [84,  8, 71]])

new_array.ravel()

array([13, 37, 42, 20, 75, 89, 84,  8, 71])
```

nw_array.ndim

newm=new_array.reshape(3,3)

newm

Output:

```
new_array.ndim

2

newm=new_array.reshape(3,3)
newm

array([[13, 37, 42],
       [20, 75, 89],
       [84,  8, 71]])
```

newm[2,1:3]

newm[1:2,1:3]

new_array[0:3,0:0]

new_array[0:2,0:1]

new_array[0:3,0:1]

new_array[1:3]

Output:

```
newm[2,1:3]

array([ 8, 71])

newm[1:2,1:3]

array([[75, 89]])

new_array[0:3,0:0]

array([], shape=(3, 0), dtype=int64)

new_array[0:2,0:1]

array([[13],
       [20]])
```

```
new_array[0:3,0:1]

array([[13],
       [20],
       [84]])

new_array[1:3]

array([[20, 75, 89],
       [84,  8, 71]])
```

Exp No:11 Pandas Library Use Cases in Data Science.

Program:

```
import numpy as
np import pandas
as pd

list=[[1,'Smith',50000],[2,'
Jones',60000]]
df=pd.DataFrame(list)
df.columns=['Empd','Name','S
alary'] df df.info()

df=pd.read_csv("/content/50_
Startups.csv") df.info()
df.head() df.tail()

df=pd.read_csv("/content/e
mployee.csv") df.head()
df.tail() df.info()

df.salary
type(df.salary)
df.salary.mean()
df.salary.median(
)
df.salary.mode()
df.salary.var()
df.salary.std()
df.describe()
df.describe(inclu
de='all')

empCol=df.columns empCol
emparray=df.values
employee_DF=pd.DataFrame(emparray,col
umns=empCol) employee_DF
```

Output:

```
Employee DataFrame output includes:
   Empd / Name / Salary    1 Smith 50000    2 Jones 60000

Employee salary
statistics:    mean
5714.285714285715
median   6000.0
mode     5000, 6000
var
2571428.5714285714
std
1603.5674514745463

Final employee_DF:
1 SREE VARSSINI K S       5000   2 SREEMATHI B             6000    3 SREYA
G                 7000    4 SREYASKARI MULLAPUDI    5000    5 SRI AKASH U
G           8000    6 SRI HARSHAVARDHANAN R   3000    7 SRI
HARSHAVARDHANAN R   6000
```

Exp No:12

Experiment to detect outliers in a given data set.

Program:

import numpy as np

array=np.random.randint(1,100,16)

array

Output:

```
import numpy as np
array=np.random.randint(1,100,9)
array
```

```
array([73, 62, 71, 78, 48, 34, 79, 58,  2])
```

```
array.mean()
```

```
np.float64(56.111111111111114)
```

```
np.percentile(array,25)
```

```
np.float64(48.0)
```

```
np.percentile(array,50)
```

```
np.float64(62.0)
```

```
np.percentile(array,75)
```

```
np.float64(73.0)
```

```
np.percentile(array,100)
```

```
np.float64(79.0)
```

Program:

```
def outDetection(array):
sorted(array)
Q1,Q3=np.percentile(array,[25,75])
IQR=Q3-Q1
lr=Q1-(1.5*IQR)
ur=Q3+(1.5*IQR)
return lr,ur
lr,ur=outDetection(array)
lr,ur
```

Output:

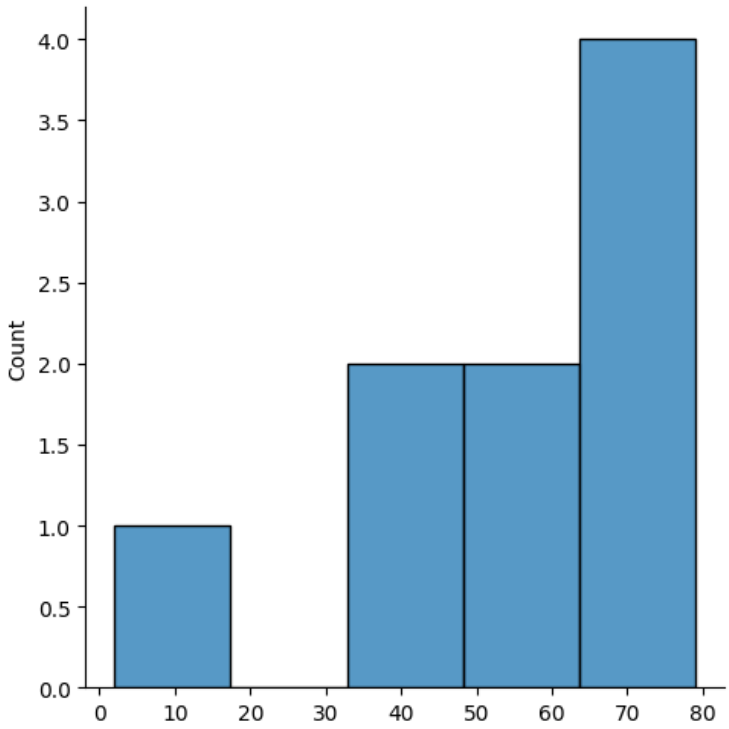
```
def outDetection(array):
  sorted(array)
  Q1,Q3=np.percentile(array,[25,75])
  IQR=Q3-Q1
  lr=Q1-(1.5*IQR)
  ur=Q3+(1.5*IQR)
  return lr,ur

lr,ur=outDetection(array)
lr,ur
```

```
(np.float64(10.5), np.float64(110.5))
```

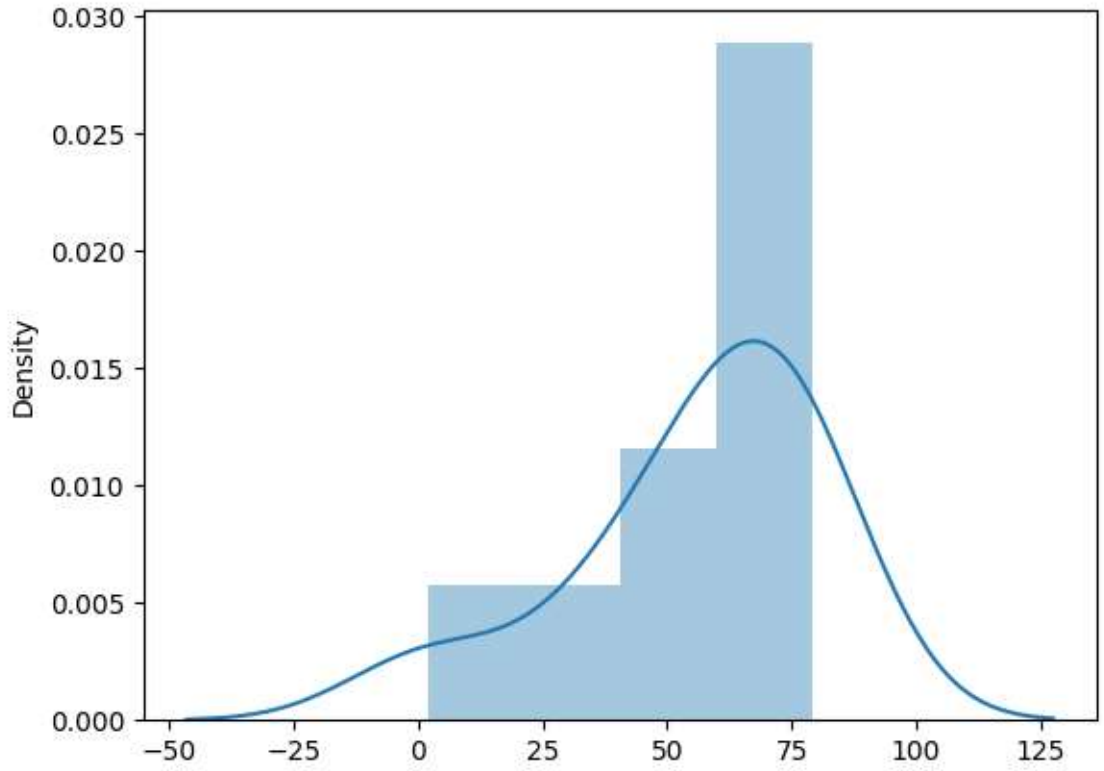
Program:

```
import seaborn as sns
%matplotlib inline
sns.displot(array)
```



sns.distplot(array)

Output:

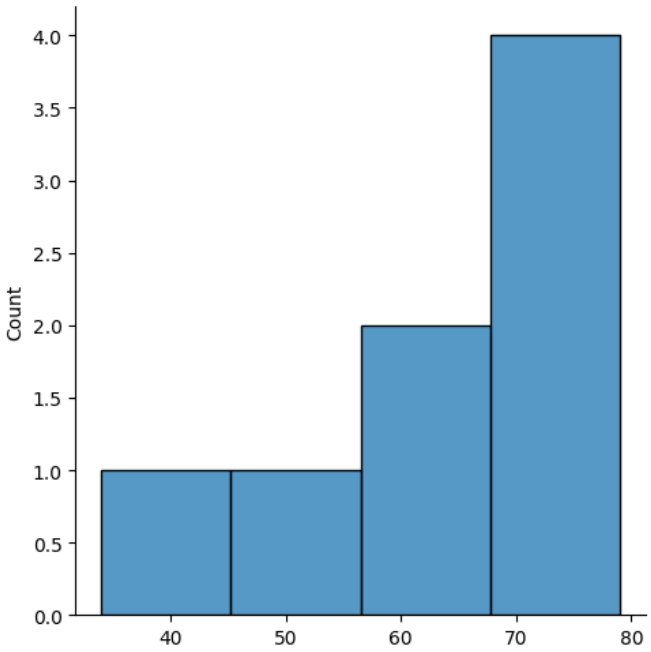


new_array=array[(array>lr) & (array<ur)]

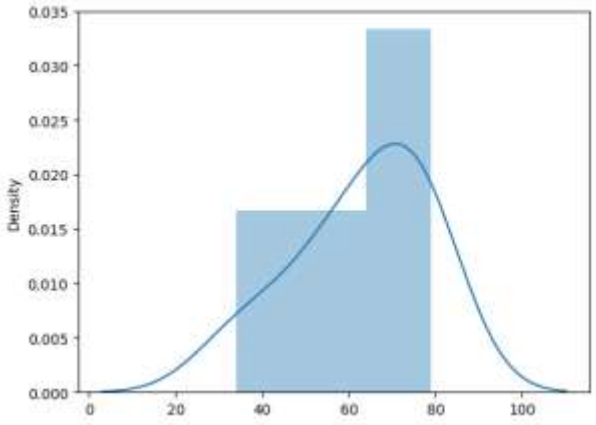
new_array

sns.displot(new_array)

Output:



sns.distplot(final_array)



Exp No:13

Experiment to handle missing values and inappropriate data in a

given data set

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv("Hotel_Dataset.csv")
df
```

Output:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

Program:



df.info()

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   CustomerID       11 non-null     int64 
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64 
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64 
 6   NoOfPax          11 non-null     int64 
 7   EstimatedSalary  11 non-null     int64 
 8   Age_Group_2      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
```

df.drop_duplicates(inplace=True)

df

```
df.drop_duplicates(inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group_2
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	6755	4	87777	30-35

len(df)

Index=np.array(list(range(0,len(df))))

df.set_index(index,inplace=True)

index

```
len(df)

10

index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index

array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

df

Output:

```
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group_2
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	10	30-35	5	RedFox	non-Veg	6755	4	87777	30-35

df.drop(['Age_Group.1'],axis=1,inplace=True)

df

Output:

```
df.drop(['Age_Group_1'],axis=1,inplace=True,errors='ignore')
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	6755	4	87777

df.CustomerID.loc[df.CustomerID<0]=np.nan

df.Bill.loc[df.Bill<0]=np.nan

df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan

df

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	veg	1300.0	2	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2	120000.0
4	5.0	35+	3	Ibis	Vegetarian	989.0	2	45000.0
5	6.0	35+	3	Ibys	Non-Veg	1909.0	2	122220.0
6	7.0	35+	4	RedFox	Vegetarian	1000.0	-1	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	-10	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3	NaN
9	10.0	30-35	5	RedFox	non-Veg	6755.0	4	87777.0

df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan

df

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	veg	1300.0	2.0	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Vegetarian	989.0	2.0	45000.0
5	6.0	35+	3	Ibys	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Vegetarian	1000.0	NaN	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	NaN	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	NaN
9	10.0	30-35	5	RedFox	non-Veg	6755.0	4.0	87777.0

df.Age_Group.unique()

df.Hotel.unique()

```
df.Age_Group.unique()

array(['20-25', '30-35', '25-30', '35+'], dtype=object)

df.Hotel.unique()

array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)
```

Program:

df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)

df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)

df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)

df.Bill.fillna(round(df.Bill.mean()),inplace=True)

df

Output:

```
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	veg	1300.0	2.0	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Vegetarian	989.0	2.0	45000.0
5	6.0	35+	3	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Vegetarian	1000.0	2.0	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5	RedFox	non-Veg	6755.0	4.0	87777.0

Exp no :14 Experiment to understand Feature Scaling

Program:

```
import numpy as np
import pandas as pd

df = pd.read_csv('pre-
process_datasample.csv')
print("Original Dataset:") print(df) print()

df['Country'] = df.Country.fillna(df.Country.mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

from sklearn.impute import SimpleImputer age_imputer =
SimpleImputer(strategy="mean", missing_values=np.nan)
salary_imputer = SimpleImputer(strategy="mean",
missing_values=np.nan) age_imputer.fit(features[:, [1]])
salary_imputer.fit(features[:, [2]]) features[:, [1]] =
age_imputer.transform(features[:, [1]]) features[:, [2]] =
salary_imputer.transform(features[:, [2]]) print("Features after
handling missing values:") print(features) print()

from sklearn.preprocessing import
OneHotEncoder oh =
OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:,
[0]]) print("One-Hot Encoded Country:")
print(Country) print()

final_set = np.concatenate((Country, features[:, [1, 2]]),
axis=1) print("Final feature set (before scaling):")
print(final_set) print()

from sklearn.preprocessing import
StandardScaler sc = StandardScaler()
sc.fit(final_set) feat_standard_scaler =
sc.transform(final_set) print("Feature set
after Standard Scaling:")
print(feat_standard_scaler) print()

from sklearn.preprocessing import
MinMaxScaler mms =
MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set) feat_minmax_scaler =
mms.transform(final_set) print("Feature
set after Min-Max Scaling:")
print(feat_minmax_scaler)
```

Output:

Original Dataset:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

Features after handling missing values:

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

One-Hot Encoded Country:

```
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
[0.  1. 0.]
[1.  0. 0.]
[0.  0. 1.]
[1.  0. 0.]
[0.  1. 0.]
[1.  0. 0.]]
```

Final feature set (before scaling):

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
```

```
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]

Feature set after Standard Scaling:
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01
7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01    1.52752523e+00 -
1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-0   1.52752523e+00 -6.54653671e-01 -
1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01   1.52752523e+00 -
1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01   1.52752523e+00 -6.54653671e-01
1.77608893e-01
   6.63219199e-16]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -
5.48972942e-01
  -5.26656882e-01]
 [-8.16496581e-01 -6.54653671e-01   1.52752523e+00
0.00000000e+00
  -1.07356980e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01
1.34013983e+00
   1.38753832e+00]
 [-8.16496581e-01   1.52752523e+00 -6.54653671e-01
1.63077256e+00
   1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -
2.58340208e-01
   2.93712492e-01]]

Feature set after Min-Max Scaling:
[[1.        0.        0.        0.73913043 0.68571429]
 [0.        0.        1.        0.        0.        1
 [0.        1.        0.        0.13043478 0.17142857]
 [0.        0.        1.        0.47826087 0.37142857]
                1.        0.        0.56521739 0.45079365]
[0.            0.        0.        0.34782609 0.28571429]
[0.            0.        1.        0.51207729 0.11428571]
[1.            0.        0.        0.91304348 0.88571429]
[0.            1.        0.        1.        1.        1
[1.            0.        0.        0.43478261 0.54285714]]
```

Exp No:15 Experiment to understand data preprocessing in Data Science.

Program:

```
import numpy as np
import pandas as pd

df = pd.read_csv("pre-process_datasample.csv")
print("Original Dataset:") print(df) print()

print("Dataset Info:")
df.info() print()

print("Mode of Country column:")
print(df.Country.mode()) print()



df['Country'] = df.Country.fillna(df.Country.mode()[0])
df['Age'] = df.Age.fillna(df.Age.median()) df['Salary']
= df.Salary.fillna(round(df.Salary.mean()))
print("Dataset after handling missing values:")
print(df) print()

print("One-Hot Encoded Country column:")
print(pd.get_dummies(df.Country))
print()

updated_dataset = pd.concat([pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
axis=1) print("Updated dataset after One-Hot Encoding:") print(updated_dataset)
print() print("Updated Dataset Info:") updated_dataset.info() print()

updated_dataset['Purchased'] = updated_dataset.Purchased.replace(['No', 'Yes'],
[0, 1]) print("Final Preprocessed Dataset:") print(updated_dataset)
```

Output

```
Original Dataset:
   Country   Age   Salary Purchased
0    France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2    Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4    Germany  40.0      NaN       Yes
5    France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7    France  48.0  79000.0       Yes
8    Germany  50.0  83000.0        No
9    France  37.0  67000.0       Yes

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9 Data
columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
0   Country    10 non-null     object
1   Age        9 non-null      float64
2   Salary     9 non-null      float64 3   Purchased  10 non-null     object dtypes: float64(2), object(2) memory usage:
    452.0+ bytes

Mode of Country column:
0    France
Name: Country, dtype: object

Dataset after handling missing values:
   Country   Age   Salary Purchased
0    France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2    Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4    Germany  40.0  63778.0       Yes
5    France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7    France  48.0  79000.0       Yes
8    Germany  50.0  83000.0        No
9    France  37.0  67000.0       Yes

One-Hot Encoded Country column:
   France  Germany  Spain
```

```
0    True    False  False
1    False    False   True
2    False     True  False
3    False    False   True
4    False     True  False
5    True    False  False
6    False    False   True
7    True    False  False
8    False     True  False
9    True    False  False

Updated dataset after One-Hot Encoding:
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1    False    False   True  27.0  48000.0       Yes
2    False     True  False  30.0  54000.0        No
3    False    False   True  38.0  61000.0        No
4    False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6    False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
8    False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes

Updated Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9 Data
columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
0   France     10 non-null     bool
1   Germany    10 non-null     bool
2   Spain      10 non-null     bool
3   Age        10 non-null     float64
4   Salary     10 non-null     float64 5   Purchased  10 non-null     object dtypes: bool(3), float64(2), object(1)
    memory usage: 402.0+ bytes

Final Preprocessed Dataset:
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1    False    False   True  27.0  48000.0          1
2    False     True  False  30.0  54000.0          0
3    False    False   True  38.0  61000.0          0
4    False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6    False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8    False     True  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
```

Output:

```
C:\Users\vsros\AppData\Local\Temp\ipykernel_20308\1860359933.py:40: FutureWarning: Downcasting behavior in `replace` is
updated_dataset['Purchased'] = updated_dataset.Purchased.replace(['No', 'Yes'], [0,
```

Exp No:16

Experiment to understand EDA-Quantitative and Qualitative analysis.

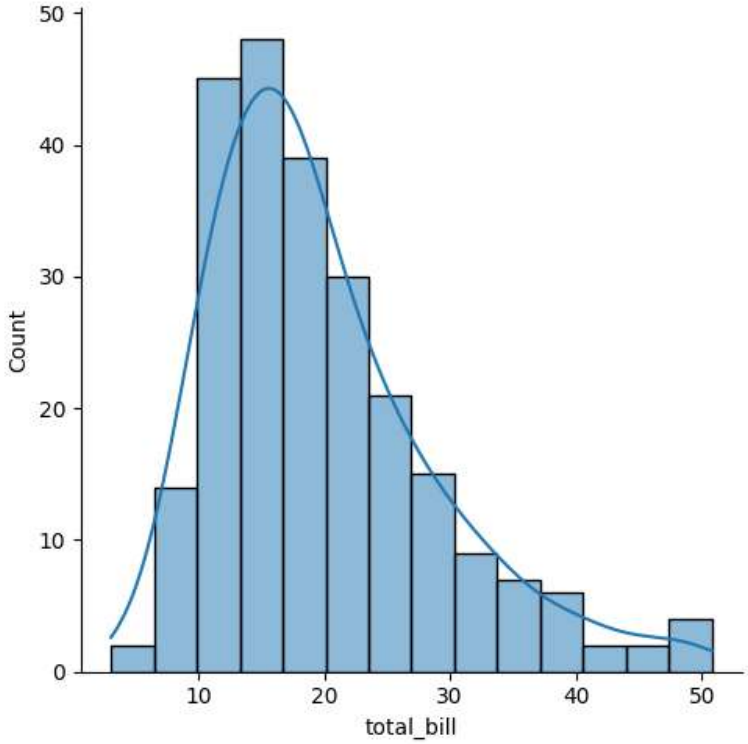
Program:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
tips.head()
```

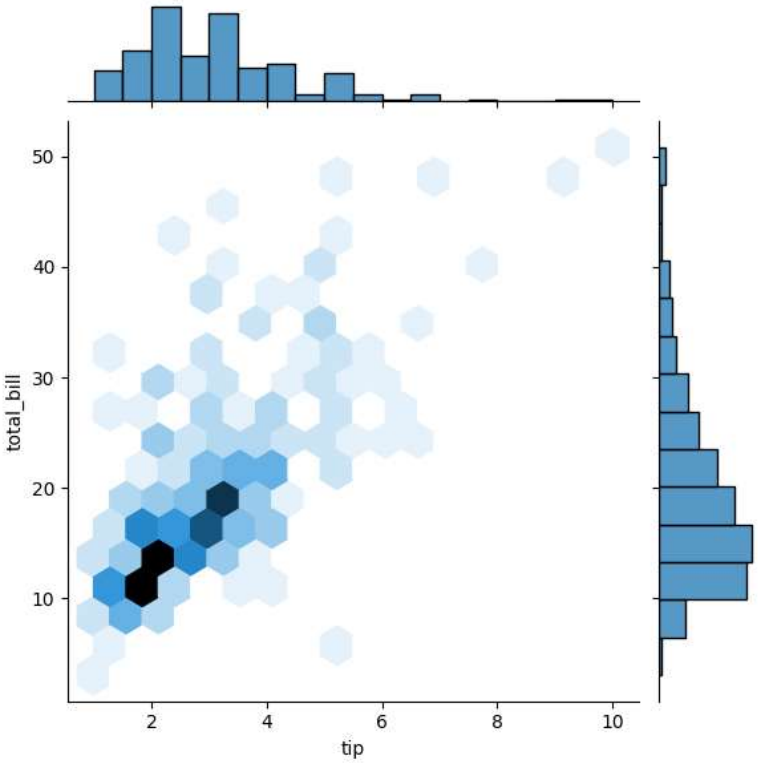
Output:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

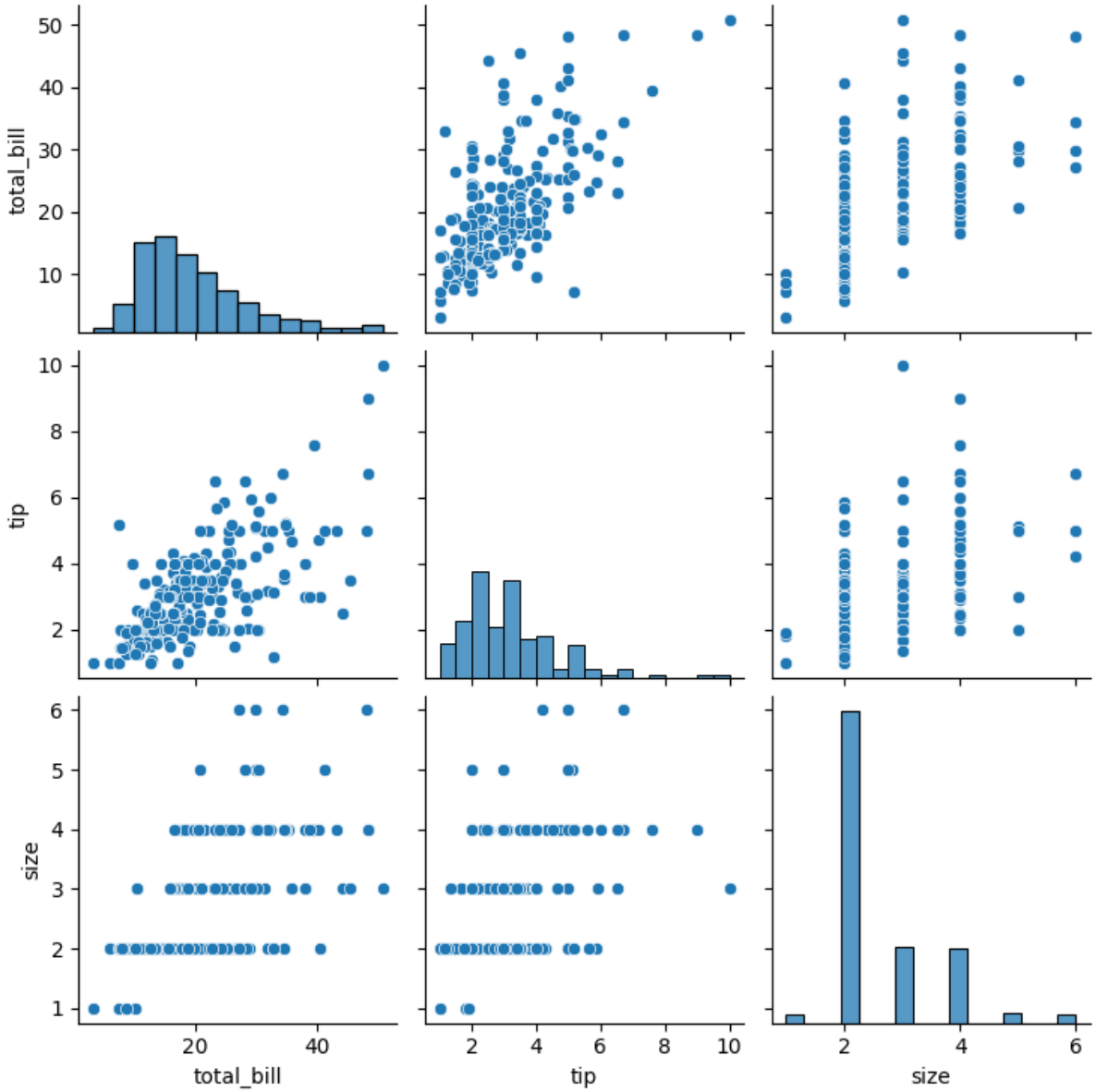
sns.displot(tips.total_bill,kde=True)



sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")



sns.pairplot(tips)

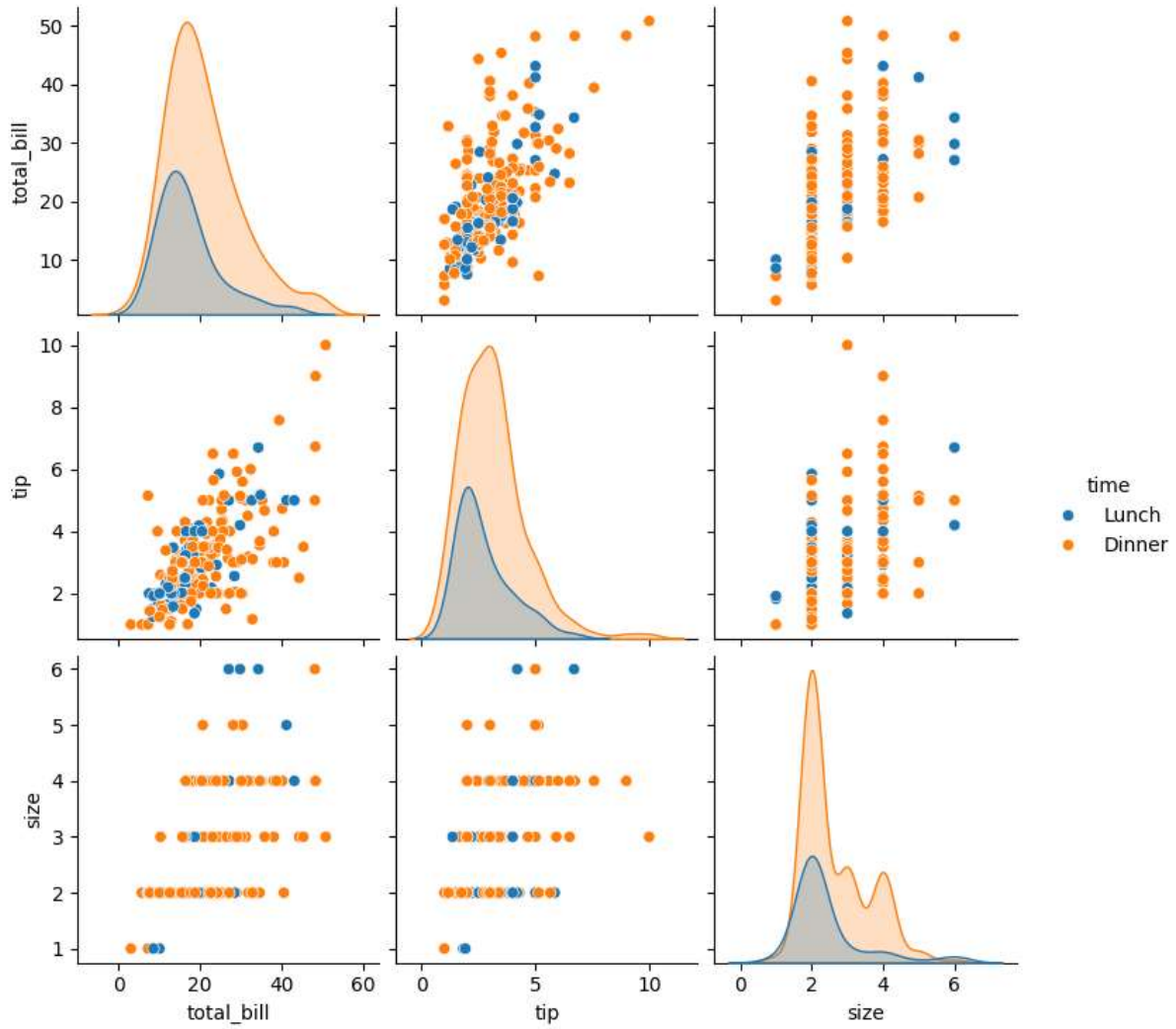


tips.time.value_counts()

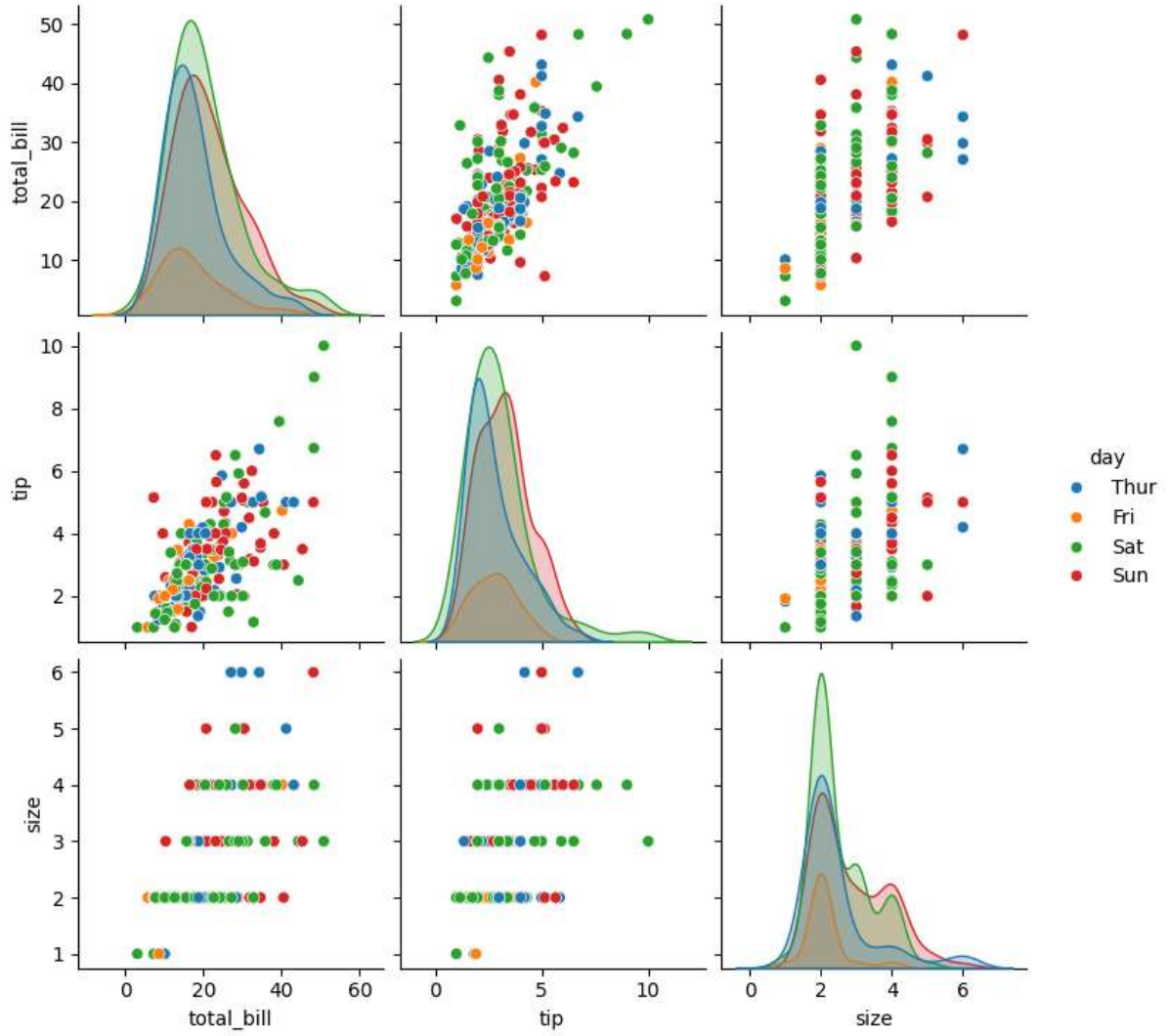
time	count
Dinner	176
Lunch	68

dtype: int64

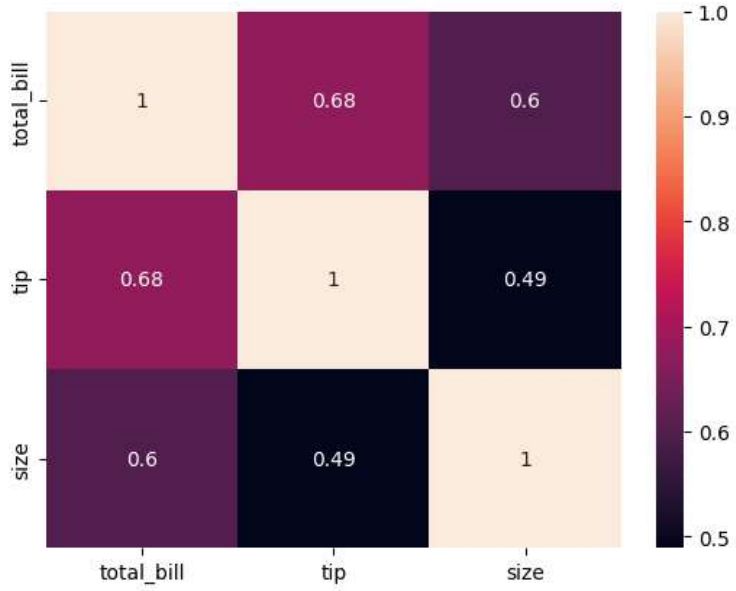
sns.pairplot(tips,hue='time')



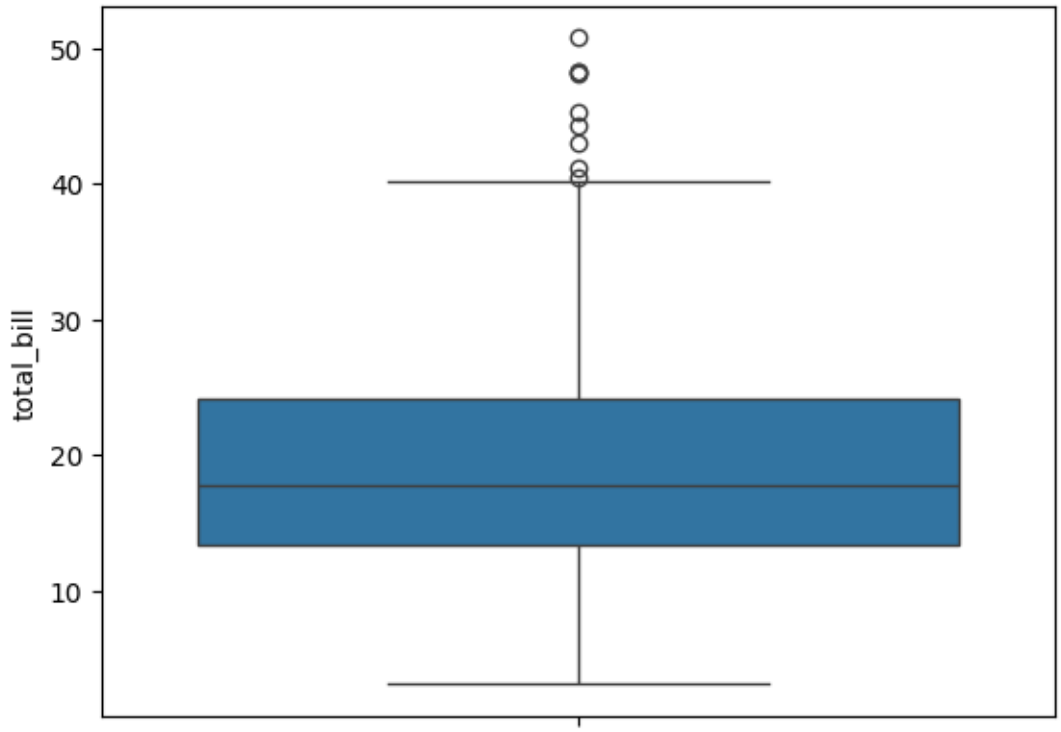
sns.pairplot(tips,hue='day')



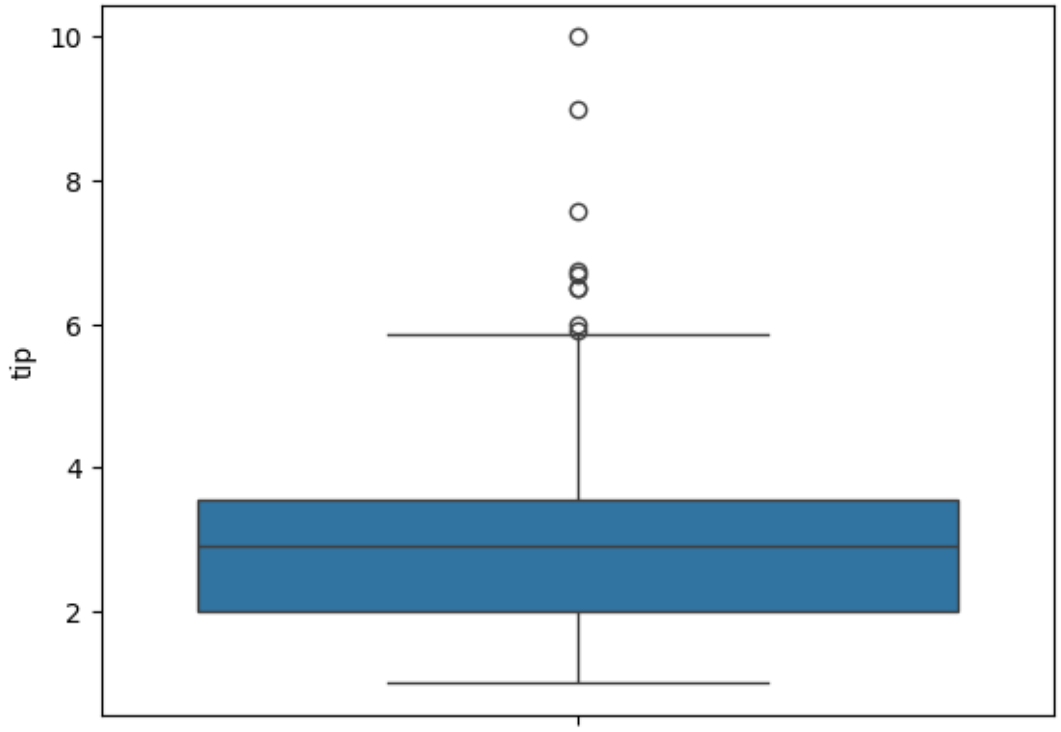
sns.heatmap(tips.corr(numeric_only=True),annot=True)



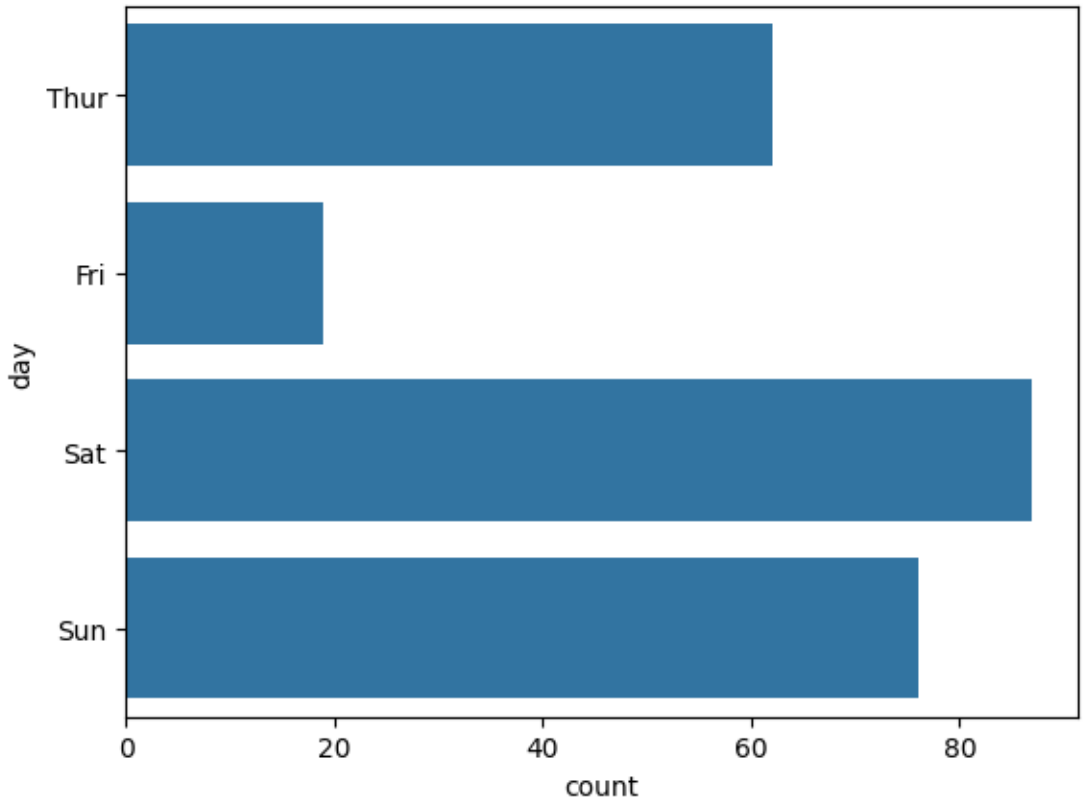
sns.boxplot(tips.total_bill)



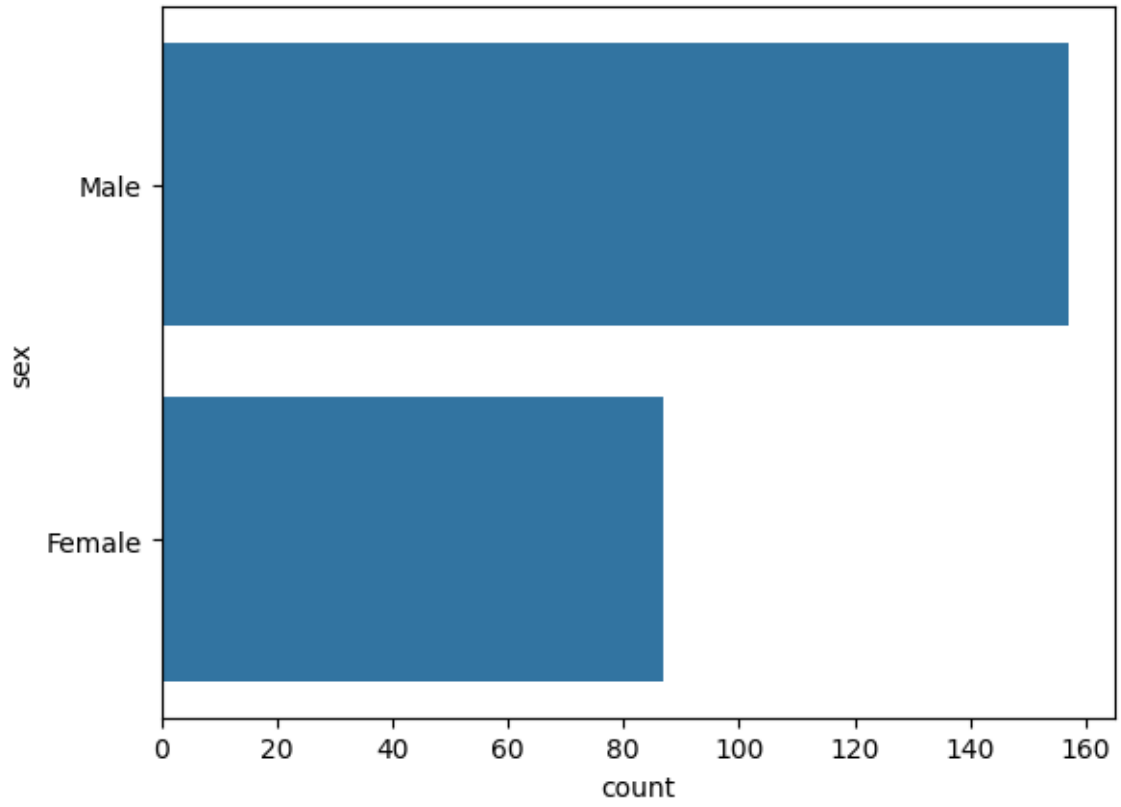
sns.boxplot(tips.tip)



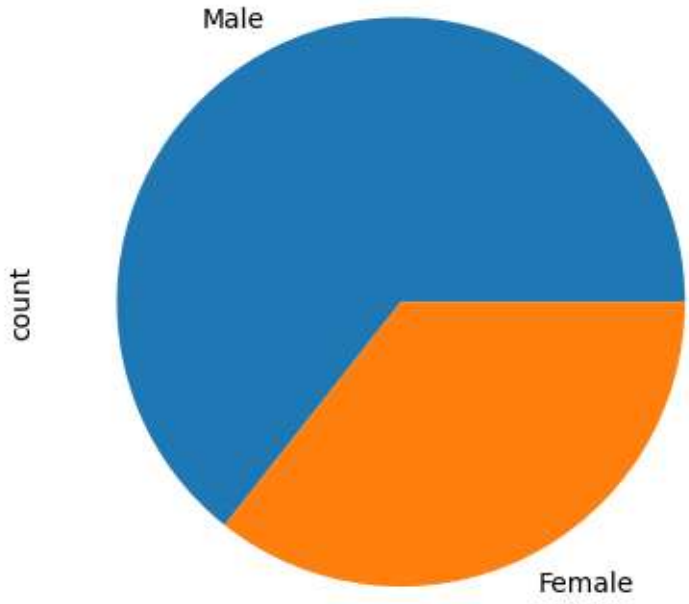
sns.countplot(tips.day)



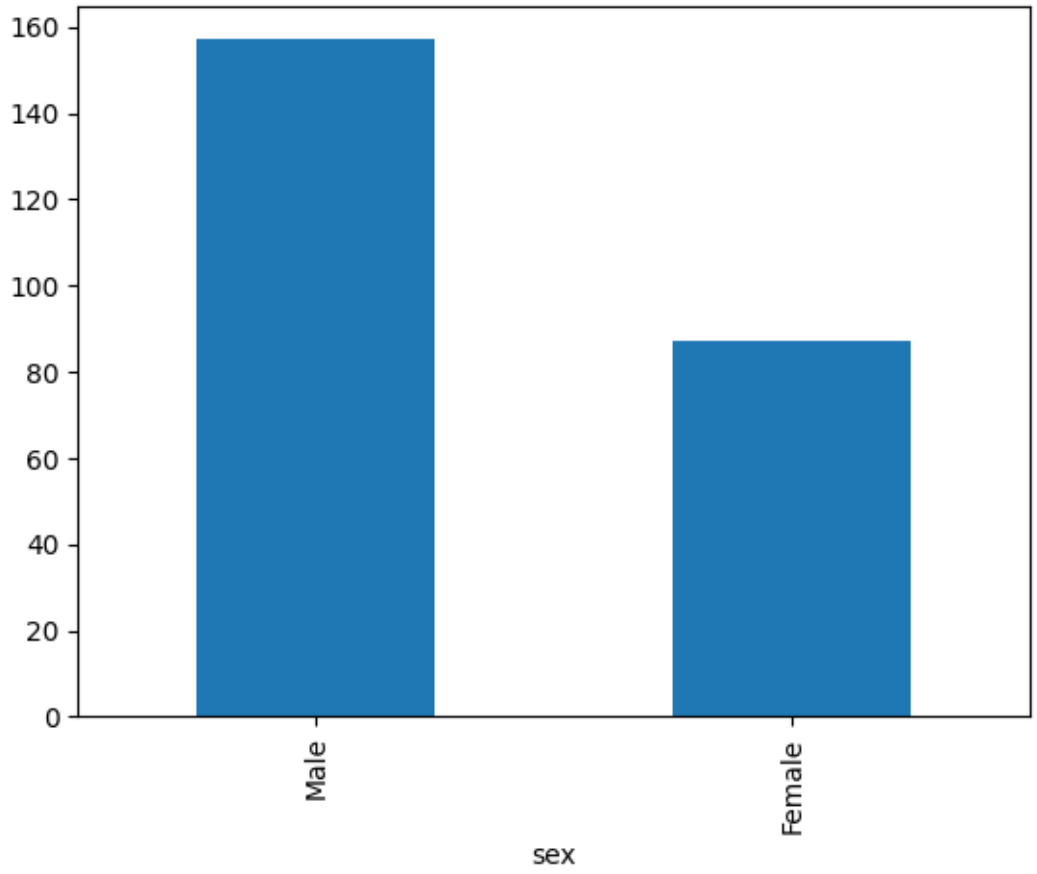
sns.countplot(tips.sex)



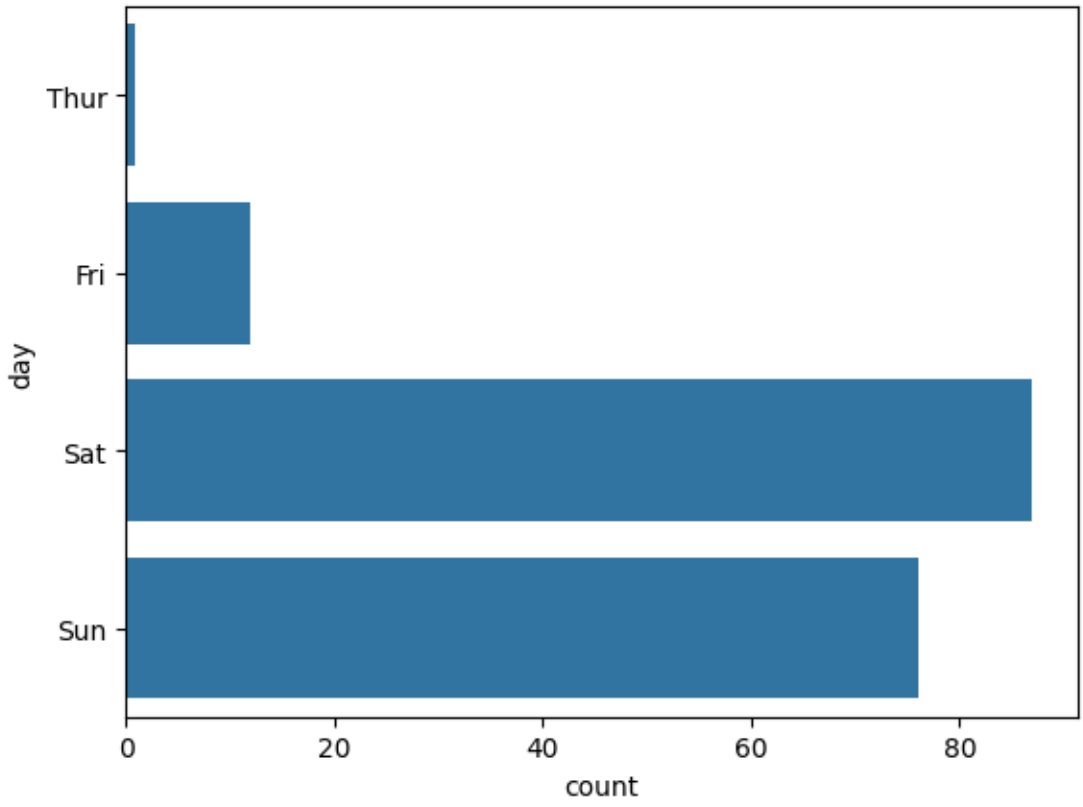
tips.sex.value_counts().plot(kind='pie')



tips.sex.value_counts().plot(kind='bar')



```
sns.countplot(tips[tips.time=='Dinner']['day'])
```



Exp No: 17

Linear Regression.

Program:

```
import numpy as np import pandas as pd
df=pd.read_csv('Salary_data.csv') df df.info()
df.dropna(inplace=True) df.info() df.describe()
features=df.iloc[:,[0]].values label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,l
abel, test_size=0.2,random_state=23) from
sklearn.linear_model import LinearRegression
model=LinearRegression() model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test) model.coef_
model.intercept_ import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP) print("Estimated
Salary for {} years of experience is {}: "
.format(yr_of_exp,Salary))
```

Output:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

```
<class
'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to
29 Data columns (total 2
columns):
 #  Column          Non-Null Count
Dtype
---  ------          --------------  -----
 0  YearsExperience 30 non-null
float64  1  Salary          30 non-null
int64 dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

Output:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

LinearRegression()

0.960318254743891

0.918417084921423

array([[9281.30847068078]])

array([27166.73682891278])

Output:

Enter Years of Experience: 44 Estimated Salary for 44.0 years of experience is [[435544.30953887]]:

Exp NO: 18

K-Nearest Neighbors (KNN).

Program:

```
import numpy as np import
pandas as pd
df=pd.read_csv('Iris.csv')
df.info()
df.variety.value_counts()
df.head()

features=df.iloc[:,:-1].values label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split from
sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(
features,label,test_size=.2,random_state=1)

model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)

print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149 Data
columns (total 5 columns):
0   sepal.length   150 non-null    float64
1   sepal.width    150 non-null    float64
2   petal.length   150 non-null    float64
3   petal.width    150 non-null    float64 4   variety        150
    non-null    object dtypes: float64(4), object(1) memory
    usage: 6.0+ KB

variety Setosa
50
Versicolor    50
Virginica     50

   sepal.length  sepal.width  petal.length  petal.width variety 0
5.1          3.5           1.4           0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
```

Scores:

```
0.9583333333333334
1.0
```

Confusion Matrix:

```
[[50  0  0]
 [ 0 47  3]
 [ 0  2 48]]
```

Classification Report:

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
macro avg       0.97      0.97      0.97       150 weighted
avg       0.97      0.97      0.97       150
```

Exp No:19

Logistic Regression.

Program:

```
import numpy as np import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv
') df df.head()

features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(
features,label,test_size=0.2,random_state=i)
model=LogisticRegression()
model.fit(x_train,y_train)
train_score=model.score(x_train,y_train)
test_score=model.score(x_test,y_test)     if
test_score>train_score:
        print("Test {} Train{} Random State
{}".format(             test_score,train_score,i))

x_train,x_test,y_train,y_test=train_test_split(
features,label,test_size=0.2,random_state=37)

finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

Output:

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0

400 rows × 5 columns

   User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510  Male     19  19000             0
1  15810944  Male     35  20000             0
2  15668575  Female   26  43000             0
3  15603246  Female   27  57000             0
4  15804002  Male     19  76000             0
```

Output:

```
Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
Test 0.675 Train0.634375 Random State 8
Test 0.65 Train0.640625 Random State 10
Test 0.6625 Train0.6375 Random State 11
Test 0.7125 Train0.625 Random State 13
Test 0.675 Train0.634375 Random State 16
Test 0.7 Train0.628125 Random State 17
Test 0.7 Train0.628125 Random State 21
Test 0.65 Train0.640625 Random State 24
Test 0.6625 Train0.6375 Random State 25
Test 0.75 Train0.615625 Random State 26
Test 0.675 Train0.634375 Random State 27
Test 0.7 Train0.628125 Random State 28
Test 0.6875 Train0.63125 Random State 29
Test 0.6875 Train0.63125 Random State 31
Test 0.6625 Train0.6375 Random State 37

LogisticRegression()

0.865625

0.8125               precision    recall  f1-score

support

           0       0.85      0.93      0.89       257
1       0.84      0.71      0.77       143

    accuracy                           0.85       400
macro avg       0.85      0.82      0.83       400
weighted avg       0.85      0.85      0.85       400
```

Exp No:20 K-Means Clustering.

Program:

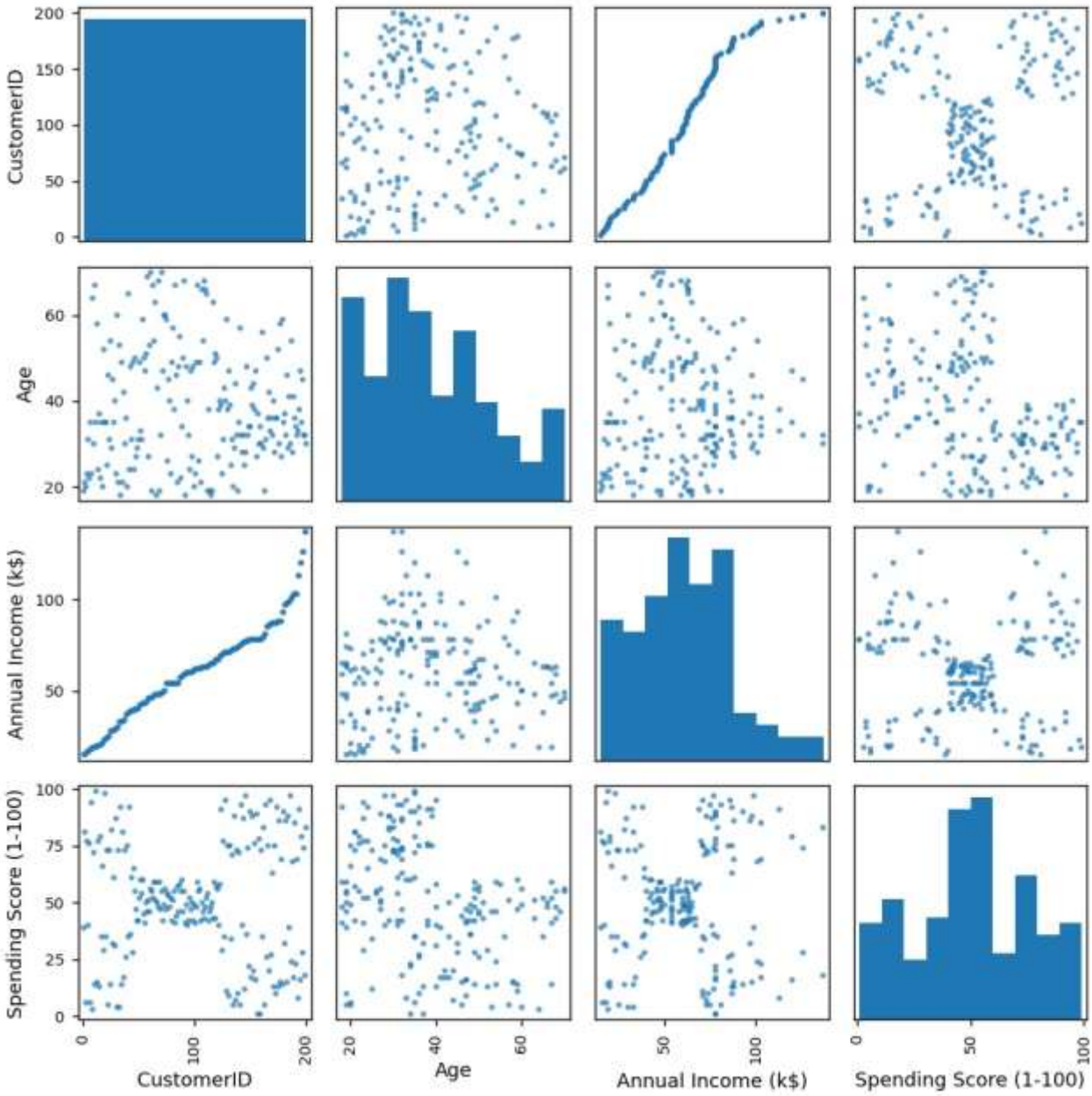
```
import numpy as np
import pandas as pd
import
matplotlib.pyplot as
plt import seaborn as
sns %matplotlib inline
df=pd.read_csv('Mall_C
ustomers.csv')
df.info() df.head()
sns.pairplot(df)
features=df.iloc[:,[3,
4]].values from
sklearn.cluster import
KMeans
model=KMeans(n_cluster
s=5)
model.fit(features)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head() sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8)
\ .map(plt.scatter,"Annual Income (k$)",
"Spending Score (1-100)") \ .add_legend()
plt.show()
features_el=df.iloc[:,[2,3,4]].values from
sklearn.cluster import KMeans wcss=[] for
i in range(1,10):
model=KMeans(n_clusters=i)
model.fit(features_el)
wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200
entries, 0 to 199
Data columns (total
5columns):
0 CustomerID              200 non-null    int64
1 Gender                  200 non-null    object
2 Age                     200 non-null    int64
3 Annual Income (k$)      200 non-null    int64 4   Spending Score
(1-100)  200 non-null    int64 dtypes: int64(4), object(1) memory
usage: 7.9+ KB
```

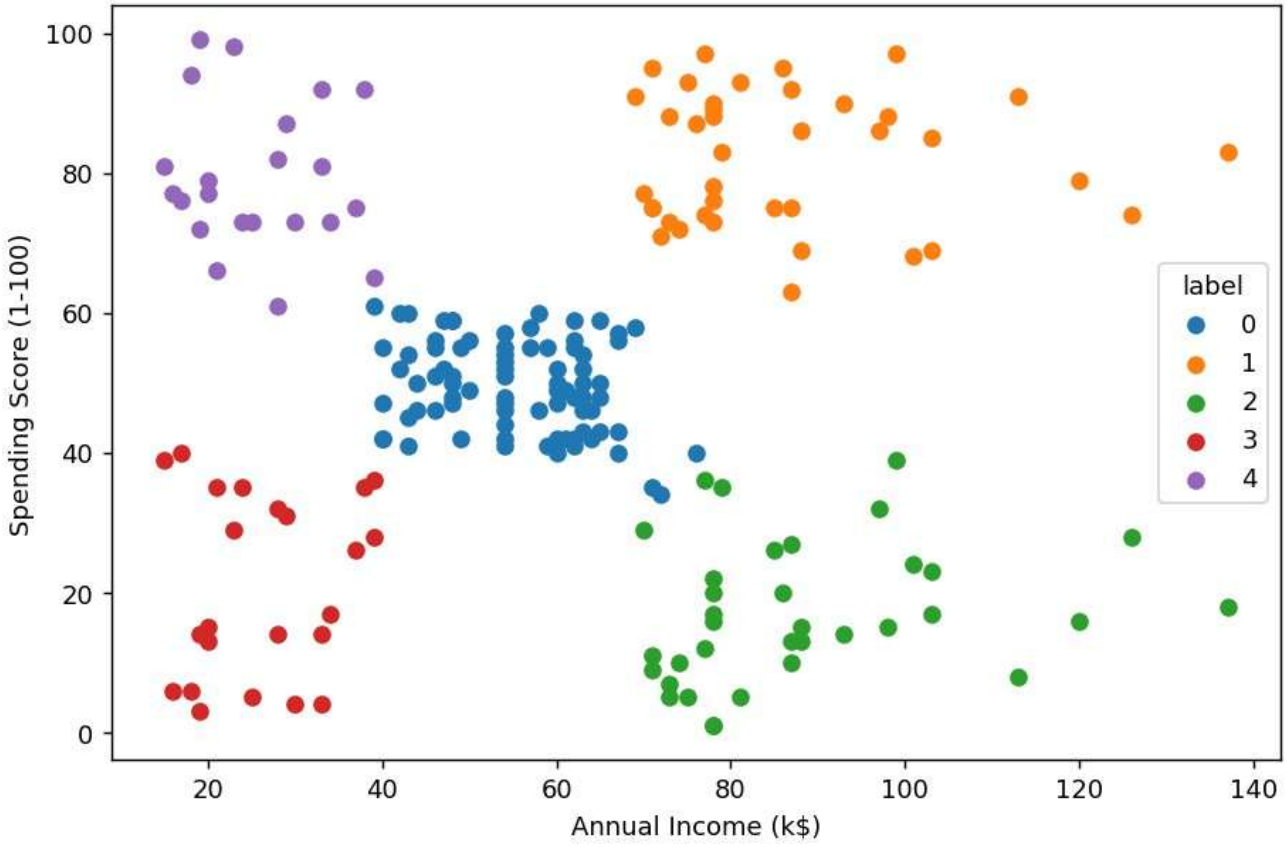
```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-
100)
```

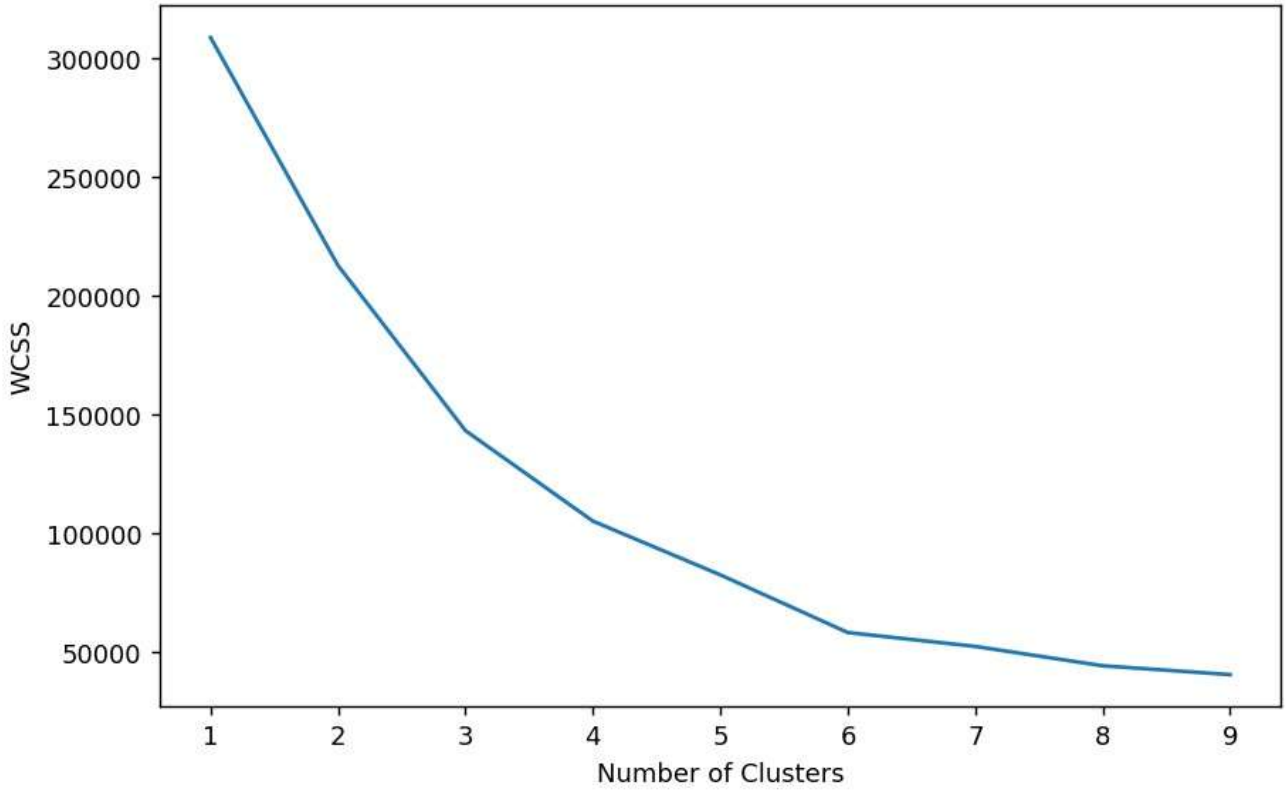
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)
0
50
100
150
200
20
40
60
25
75

Output

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	3
1	15	81	4
2	16	6	3
3	16	77	4
4	17	40	3


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4
100
80
60
40
20
0
20
40
60
80
100
120
140


WCSS
Number of Clusters
300000
250000
200000
150000
100000
50000
1
2
3
4
5
6
7
8
9